Catalogue des sujets de projet

OliveSmart Maroc (7 sujets) + SmartAI Systems IIR5 (7 sujets)

Partie A — Socles communs : A1 OliveSmart

A1.1 Vision d'ensemble

Les sept travaux sont des modules scientifiques autonomes d'une même plateforme de gestion intelligente des oliveraies marocaines. Les services métiers, la sécurité et les API sont développés en Java/Spring Boot ; les modèles d'IA sont entraînés en Python et exposés par un service d'inférence FastAPI/MLflow ou par des modèles ONNX. Les données géospatiales sont stockées dans PostgreSQL/PostGIS et les images dans MinIO.

Une séparation explicite entre apprentissage des modèles et consommation des prédictions limite les couplages. L'entraînement est asynchrone ; les résultats d'inférence sont historisés avec leur version de modèle et de dataset.

A1.2 Microservices du socle partagé

Service commun	Technologie et fonction
api-gateway	Spring Cloud Gateway : routage, sécurité, limitation de débit.
identity-service	Spring Security / OAuth2-OIDC (Keycloak) : utilisateurs, rôles et accès aux parcelles.
farm-service (orchard-tree-service)	Spring Boot + PostGIS : exploitations, parcelles, arbres, campagnes.
media-service	Spring Boot + MinIO : upload, métadonnées, URLs signées.
dataset-service	Spring Boot : jeux de données, splits, licences et versions.
model-registry-service	Spring Boot + MLflow : versions, métriques et approbation des modèles.
job-service	Spring Boot + RabbitMQ/Kafka : tâches longues, statuts, reprises.
notification-service	Spring Boot : alertes, notifications et acquittements.
audit-observability	Actuator, OpenTelemetry, Prometheus/Grafana : traçabilité et mesures.

A1.3 Architecture microservices commune

Clients web/mobile → API Gateway → services Spring Boot → PostgreSQL/PostGIS et MinIO. Pour l'IA : service métier → file de messages → worker Python (prétraitement / inférence) → résultat normalisé → base de données → notification et tableau de bord. Aucun fichier volumineux ne circule dans la file : seul son identifiant ou son URI interne y est transmis.

Élément	Description
Clients	Application web, mobile et tableau de bord agronomique.
Entrée sécurisée	Spring Cloud Gateway + Keycloak : routage, authentification, autorisation et limitation de débit.
Sept modules métier	Tree Detection (S1), Disease Diagnosis (S2), Irrigation (S3), Satellite Analysis (S4), Yield Prediction (S5), Tree Health Monitoring (S6), Variety Classification (S7). La gestion des parcelles et arbres est mutualisée dans Orchard Service.
Exécution IA	Chaque service Spring Boot orchestre les traitements et sollicite un moteur Python/FastAPI versionné ; les tâches longues passent par une file de messages.
Données et événements	PostgreSQL/PostGIS, stockage objet MinIO, RabbitMQ ou Kafka, MLflow pour les modèles.
Socle DevOps	CI/CD, conteneurisation, Kubernetes (optionnel), observabilité, résilience et MLOps.
Socle DevSecOps	Contrôles SAST/SCA/DAST, SBOM, signature des artefacts, secrets, IAM, mTLS, politiques Kubernetes et audit des API.

Conventions de conception

- Chaque service possède son schéma ou sa base logique ; communication interservices par API ou événements, sans accès direct aux tables privées des autres services.

- REST synchrone pour les consultations rapides ; événement asynchrone pour orthomosaïques, séries satellites, entraînement, inférence massive et rapports.
- Métadonnées communes : exploitation_id, parcelle_id, tree_id (si pertinent), date_observation, source, dataset_version, model_version, confidence, statut_validation.
- Spring Boot orchestre les processus mais n'entraîne pas les réseaux neuronaux dans la JVM ; conserver des workers Python isolés.
- Docker Compose suffit pour un ; Kubernetes, service discovery dédié et circuit breakers avancés sont des extensions si la charge l'exige.

A1.4 Schéma technique cible

Couche	Décision de conception
Frontend	React ou Next.js, cartographie Leaflet/OpenLayers, vues par parcelle et par arbre.
API	Spring Boot, Spring Cloud Gateway, sécurité JWT/OIDC.
Communication	REST pour requêtes courtes ; RabbitMQ ou Kafka pour jobs et événements.
Persistance	PostgreSQL + PostGIS pour les géométries ; MinIO pour les images ; Redis en cache optionnel.
IA	PyTorch / scikit-learn / XGBoost, workers Python FastAPI ou MLflow serving.
MLOps	MLflow (expérimentation, version modèle), DVC (jeux de données), CI/CD GitHub Actions.
Supervision	Spring Boot Actuator, Prometheus, Grafana ; OpenTelemetry pour les traces.
Livraison	Docker Compose pour le MVP, Kubernetes optionnel ultérieurement.

A1.5 Contrats de messages et sécurité

- Événement JobRequested : jobId, projectType, parcelId, inputUris, modelVersion, requestedBy, traceId, createdAt.
- Événement JobCompleted : jobId, status, resultUri, metrics, modelVersion, durationMs, errorCode ; traitement idempotent.
- Images par URL signées de courte durée, antivirus/contrôle MIME et droits par exploitation ; jamais d'accès public par défaut.
- Conserver version du modèle, configuration, seuils, données d'entrée et historiser la validation agronome.
- Chaque endpoint : validation DTO, gestion d'erreur structurée, authentification et permissions au niveau parcelle.

A1.6 Stratégie de comparaison scientifique commune

- Toujours utiliser une baseline simple + au moins 2 modèles ML/DL ; privilégier 3 à 5 modèles bien évalués plutôt que 10 mal entraînés.
- Séparer train, validation et test par parcelle/exploitation et, selon le cas, par campagne ; interdire les fuites spatiales et temporelles.
- Fixer graines aléatoires, versions de librairies, budget d'entraînement, GPU et métriques avant les expérimentations.
- Mesurer qualité prédictive ET coût : temps d'entraînement, latence d'inférence, RAM/VRAM, consommation CPU/GPU.
- Effectuer au moins une étude d'ablation, une analyse d'erreurs et une validation sur données marocaines nouvelles.
- Définir des seuils de non-déploiement : dérive, faible confiance, qualité de données, erreurs critiques terrain.

A1.7 Services réutilisables entre sujets

Service	Réutilisé par
tree-inventory-service / orchard-tree-service	S1 (source), S5 (comptage d'arbres), S6 (identité d'arbre), S7 (inventaire).
tree-detection-service	S1 (source), S7 (comptage YOLO/Faster R-CNN).
weather-data-service	S3 (source), S4 (fusion météo), S5 (features climat), S7 (indicateurs).
vegetation-index-service	S4 (source), S5 (features satellite), S6 (indices de vigueur), S7 (santé des cultures).
plant-health-alert-service / notification-service	S2 (source), S6 (alertes dépérissement), S7 (arbres incertains).

Socles communs : A2 SmartAI

A2.1 Cadre pédagogique

Chaque groupe d'étudiants entraîne et compare deux modèles sur une même tâche, dont une architecture avancée imposée. Les étudiants expliquent le principe et les entrées/sorties de chaque modèle, adaptent une implémentation publique, optimisent quelques paramètres et analysent les erreurs. Aucune preuve mathématique approfondie ni réimplémentation bas niveau des couches n'est exigée.

A2.2 Architecture commune

2 ou 3 microservices métier Spring Boot, un service Python FastAPI (ai-inference) pour l'inférence du modèle entraîné, une base PostgreSQL, une interface web légère et Docker Compose. REST suffit ; les files de messages ne sont utiles que pour des tâches longues réellement mises en œuvre. L'entraînement n'est pas exécuté en production par les services Spring Boot.

Élément	Description
Services métier Spring Boot	Données, historique, prévisions/résultats et alertes ; communiquent en REST avec ai-inference.
ai-inference (FastAPI)	Charge les deux modèles entraînés, applique le prétraitement identique à l'entraînement, renvoie résultat et latence.
Base de données	PostgreSQL : observations, prédictions (modèle, version, horodatage), alertes.
Interface web	Visualisation réel/prédit ou original/résultat, déclenchement des prédictions, alertes.
Livraison	Docker Compose ; documentation d'installation ; tests REST.

A2.3 Livrables communs et validation minimale

- Dépôt de code, notebook(s) reproductibles, poids des modèles, API documentée, schéma d'architecture, démonstration bout en bout, comparaison expérimentale et rapport court.
- Privilégier sous-ensembles et modèles légers compatibles avec une session Colab gratuite, sans garantie de disponibilité GPU.
- Deux entraînements réels, avec train/validation/test appropriés (temporels pour les séries), sur la même cible et des partitions comparables.
- Optimisation limitée à quelques hyperparamètres justifiés et à un budget annoncé.
- Tester toute la chaîne import → prédiction → affichage → archivage ; démarrage reproductible documenté.
- Vérifier licences, versions et accès effectif aux datasets et dépôts avant attribution des sujets.

A2.4 Vue d'ensemble des sept plateformes SmartAI

Plateforme	Modèle de référence	Architecture avancée obligatoire
AquaSmart (SAI-1)	LSTM	TCN léger
MedRestore Smart (SAI-2)	CNN résiduel de débruitage (petit DnCNN)	U-Net léger
MedVision Smart (SAI-3)	U-Net	U-Net++
EcoSense Smart (SAI-4)	GRU	Transformer temporel compact
IndustryGuard Smart (SAI-5)	CNN 1D	TCN
EnergyTwin Smart (SAI-6)	LSTM	LSTM avec couche d'attention
UrbanFlow Smart (SAI-7)	LSTM par capteur	Graph Convolution + LSTM (GCN-LSTM)

Partie B — Répartition en 3 groupes de 8 sujets

Règles appliquées : 3 groupes × 8 sujets = 24 places pour 14 sujets ; chaque sujet apparaît au moins une fois et au plus deux fois ; aucun sujet n'est répété à l'intérieur d'un même groupe. Résultat : 10 sujets figurent dans 2 groupes et 4 sujets dans 1 seul groupe.

Code	Sujet	G1	G2	G3	Nb
OLV-1	Détection, comptage et géolocalisation des oliviers par drone			●	1
OLV-2	Diagnostic précoce des maladies de l'olivier	●		●	2
OLV-3	Estimation intelligente des besoins en irrigation	●	●		2
OLV-4	Analyse satellitaire multi-temporelle des oliveraies		●	●	2
OLV-5	Prévision du rendement des oliveraies		●		1
OLV-6	Détection temporelle des oliviers dépérissants ou morts	●		●	2
OLV-7	Plateforme intelligente de comptage des oliviers et de reconnaissance de leurs variétés (IA + données géospatiales)			●	1
SAI-1	AquaSmart — Prévision intelligente des ressources hydriques	●	●		2
SAI-2	MedRestore Smart — Restauration intelligente d'images			●	1
SAI-3	MedVision Smart — Segmentation biomédicale intelligente	●		●	2
SAI-4	EcoSense Smart — Analyse intelligente de mesures environnementales	●	●		2
SAI-5	IndustryGuard Smart — Surveillance industrielle par séries temporelles	●	●		2
SAI-6	EnergyTwin Smart — Prévision intelligente de consommation électrique	●	●		2
SAI-7	UrbanFlow Smart — Prévision du trafic par réseau de graphes		●	●	2

Groupe 1 — Surveillance, alerte et aide à la décision : Sujets centrés sur une chaîne de décision : prédiction → seuil/validation humaine → alerte.

Groupe 2 — Séries temporelles et prévision : Sujets de prévision et de classification sur séries (météo, eau, énergie, trafic, signaux industriels, rendement).

Groupe 3 — Vision, imagerie et analyse spatiale : Sujets dont le cœur est l'image (drone, photo, satellite, médical) ou la structure spatiale (graphe de capteurs).

Partie C — Groupes de sujets

Groupe 1 — Surveillance, alerte et aide à la décision

Sujets centrés sur une chaîne de décision : prédiction → seuil/validation humaine → alerte.

Code	Sujet
OLV-2	1.1 — Diagnostic précoce des maladies de l'olivier
OLV-3	1.2 — Estimation intelligente des besoins en irrigation
OLV-6	1.3 — Détection temporelle des oliviers dépérissants ou morts
SAI-1	1.4 — AquaSmart — Prévision intelligente des ressources hydriques
SAI-3	1.5 — MedVision Smart — Segmentation biomédicale intelligente
SAI-4	1.6 — EcoSense Smart — Analyse intelligente de mesures environnementales
SAI-5	1.7 — IndustryGuard Smart — Surveillance industrielle par séries temporelles
SAI-6	1.8 — EnergyTwin Smart — Prévision intelligente de consommation électrique

Sujet 1.1 — [OLV-2] Diagnostic précoce des maladies de l'olivier

Sujet également présent dans le Groupe 3 (fiche identique).

1.1.1 Problématique

Classifier des symptômes phytosanitaires, localiser les lésions lorsque des masques sont disponibles et produire une alerte avec incertitude, sans remplacer l'expertise agronomique.

1.1.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
CNN olive dataset (GitHub)	https://github.com/sinanuguz/CNN_olive_dataset	Classification d'images de feuilles d'olivier saines/malades.	Document initial
Olive leaf disease (Kaggle, « zeytin »)	https://www.kaggle.com/datasets/serhathoca/zeytin	Classification de maladies foliaires ; comparaison de backbones.	Document initial
Olive tree hyperspectral stress (Zenodo)	https://zenodo.org/records/10336545	Détection de stress hydrique/biotique par signatures spectrales.	Document initial
Collecte Maroc	Données internes / à fournir	Feuille/fruit/arbre, classe validée par agronome, variété, région, saison, gravité, résolution, indicateur image_inexploitable ; masques de lésion pour la segmentation.	À constituer

1.1.3 Algorithmes à comparer

Approche	Modèle	Rôle de comparaison
Baseline	Descripteurs couleur/texture + SVM	Référence interprétable
Classification	ResNet18 / EfficientNet	Transfert léger et efficace
Classification	ConvNeXt-Tiny	Backbone convolutionnel moderne
Classification	ViT-Tiny / ViT-Small	Comparaison Transformer
Segmentation	U-Net / SegFormer	Localisation si masques disponibles

1.1.4 Optimisation

- Comparaison AdamW vs SGD momentum ; learning rate 1e-5 à 1e-3, weight decay, dégel progressif des couches, taille d'image et batch size.
- Pondération des classes, focal loss ou sampler équilibré, sans mélanger les images d'un même arbre entre splits.
- Early stopping sur macro-F1, ReduceLROnPlateau ou scheduler cosinus, warmup selon l'architecture.
- Optuna TPE / random search sur lr, dropout et augmentation ; 20–30 essais selon le budget GPU.
- Calibration des probabilités (temperature scaling) et seuil d'abstention (image inexploitable, faible confiance) défini sur validation.

1.1.5 Protocole expérimental

1. Faire relire les étiquettes par un agronome ; ne pas fusionner des maladies distinctes sous une étiquette vague.
2. Séparer entraînement/validation/test par arbre ou exploitation ; conserver une exploitation externe.
3. Comparer classification avec/sans transfer learning ; produire des matrices de confusion par classe et par variété.

4. Évaluer en conditions OOD (autres appareils, éclairages, régions) et mesurer les erreurs de confiance.
5. Valider le flux de confirmation humaine avant la création d'une alerte d'intervention.

1.1.6 Indicateurs de performance

Macro-F1, rappel par maladie, matrice de confusion, ROC-AUC si pertinent, ECE/calibration, taux d'abstention, Dice pour les lésions.

1.1.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
plant-image-service	Reçoit les photos (feuille, fruit, arbre), contrôle leur qualité (flou, exposition, résolution, MIME, antivirus), les associe à leur contexte (parcelle, variété, date) et les stocke dans MinIO. Marque les images inexploitables avant toute inférence.	Entrée : photo + contexte. Sortie : imageId, score qualité, URI MinIO.
disease-diagnosis-service	Orchestre la classification et, si besoin, la segmentation via un worker Python. Applique la calibration (temperature scaling) et le seuil d'abstention, puis renvoie top-k, probabilité calibrée, masque de lésion et statut (automatique / abstention). Historise model_version.	Entrée : imageId. Sortie : diagnostic, confiance, masque, statut.
agronomist-review-service	File de validation experte : l'agronome confirme ou corrige le diagnostic, la décision est historisée et les corrections sont renvoyées au dataset-service pour enrichir les futurs réentraînements.	Entrée : diagnosticId. Sortie : décision, annotation corrigée.
plant-health-alert-service	Crée les alertes phytosanitaires uniquement après confirmation humaine, calcule la priorité (gravité × surface × confiance), conserve l'historique par parcelle et déclenche les notifications via notification-service.	Entrée : diagnostic confirmé. Sortie : alerte priorisée, notification.

Worker IA associé (Python)

disease-inference-worker (Python / FastAPI / PyTorch) : classification, segmentation, calibration ; modèles ResNet/EfficientNet/ConvNeXt/ViT exportables en ONNX.

Exemples de contrats REST

```
POST /api/plant-images ; POST /api/disease-diagnoses
GET /api/disease-diagnoses/{id}
POST /api/agronomist-reviews/{diagnosisId}/decision
```

1.1.8 Risques et contrôles

Biais de fond/éclairage ; maladies absentes des datasets ; confusion stress/maladie ; surconfiance des scores.

1.1.9 Livrables

Classification comparée, détection des images non fiables, API d'analyse de photo, interface de validation agronome.

Sujet 1.2 — [OLV-3] Estimation intelligente des besoins en irrigation

Sujet également présent dans le Groupe 2 (fiche identique).

1.2.1 Problématique

Prédire un besoin hydrique conseillé à horizon court en respectant la sécurité agronomique, et comparer les recommandations à une méthode FAO-56 de référence.

1.2.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
NASA POWER	https://power.larc.nasa.gov/	Météo journalière (rayonnement, température, vent, humidité, pluie).	Document initial
ERA5-Land (Earth Engine)	https://developers.google.com/earth-engine/datasets/catalog/ECMWF_ERA5_LAND_DAILY_AGGR	Réanalyse climatique, humidité du sol, évapotranspiration.	Document initial
FAO WaPOR	https://www.fao.org/in-action/remote-sensing-for-water-productivity/wapor-data-access/en	Évapotranspiration et productivité de l'eau par télédétection.	Document initial
FAO-56 (Irrigation and Drainage Paper 56)	https://www.fao.org/4/x0490e/x0490e00.htm	Référence méthodologique pour ET0 (Penman-Monteith), Kc et bilan hydrique.	Ajout
SoilGrids (ISRIC)	https://soilgrids.org/	Propriétés des sols (texture, matière organique) pour la réserve utile.	Ajout
Open-Meteo	https://open-meteo.com/	Prévisions météo à 3–14 jours pour les horizons de recommandation.	Ajout
Collecte Maroc (indispensable)	Données internes / à fournir	ET0 et Kc contextualisés, pluies, sol, humidité de sondes, volumes réellement appliqués, rendement, disponibilité de l'eau.	À constituer

1.2.3 Algorithmes à comparer

Approche	Modèle	Rôle de comparaison
Référence physique	FAO-56 : ETc = Kc × ET0 + bilan hydrique	Baseline agronomique
Tabulaire	Random Forest	Robustesse simple
Tabulaire	XGBoost / CatBoost	Relations non linéaires
Temporel	LSTM / TCN	Séquences météo et humidité
Hybride	FAO-56 + XGBoost résiduel	Correction physique pilotée par les données

1.2.4 Optimisation

- RF/XGBoost : profondeur, n_estimators, learning_rate, min_child_weight, subsample ; pas d'AdamW pour les arbres de décision.
- LSTM/TCN : Adam ou AdamW, 1e-4 à 1e-3, fenêtres de 7/14/30 jours, dropout, hidden size, early stopping.
- Hybride : optimiser l'erreur résiduelle à FAO-56 ; imposer des bornes d'eau et des règles métier explicites.

- Optuna TPE ou random search avec validation temporelle en chaîne (walk-forward) ; comparaison à coût de calcul équivalent.
- Tester les horizons 3, 7 et 14 jours ; quantiles ou intervalles conformes pour l'incertitude.

1.2.5 Protocole expérimental

1. Constituer des séries par parcelle, dater les interventions, ne pas remplir artificiellement les données critiques manquantes.
2. Entraîner sur les campagnes anciennes, valider sur la période récente, tester sur la campagne suivante.
3. Comparer au minimum FAO-56, RF, XGBoost et hybride ; LSTM si les données le permettent.
4. Évaluer hors ligne, puis sur parcelles pilotes avec groupe témoin si les partenaires agronomiques l'autorisent.
5. Bloquer les recommandations lorsque sondes ou prévisions sont périmées ou incohérentes.

1.2.6 Indicateurs de performance

MAE/RMSE (volume), erreur d'humidité, m³/ha économisés, rendement, proportion de recommandations risquées.

1.2.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
weather-data-service	Collecte NASA POWER, ERA5-Land et prévisions, normalise les unités, le pas de temps et le fuseau, gère le cache et la qualité des séries, et expose observations et prévisions par parcelle.	Entrée : parcelle, période. Sortie : séries météo normalisées.
soil-sensor-service	Ingestion IoT (HTTP/MQTT) des sondes d'humidité, calibration, détection des sondes périmées ou incohérentes, séries d'humidité par profondeur et par parcelle.	Entrée : mesures capteurs. Sortie : séries d'humidité, indicateur de fiabilité.
water-balance-service	Calcule ET0 (Penman-Monteith FAO-56), ETc = Kc × ET0 et le bilan hydrique (pluie efficace, réserve utile, drainage) par parcelle ; fournit la baseline physique.	Entrée : météo, sol, Kc. Sortie : ET0, ETc, déficit hydrique.
irrigation-recommendation-service	Appelle le worker (RF, XGBoost, LSTM ou hybride), applique les règles de sécurité (bornes, blocage si données périmées), produit dose, planning et intervalle d'incertitude à 3/7/14 jours.	Entrée : parcelle, horizon. Sortie : recommandation, planning, incertitude.
irrigation-tracking-service	Enregistre les volumes réellement appliqués, compare recommandation et réalité, calcule les m³ économisés et alimente le réentraînement.	Entrée : événements d'irrigation. Sortie : historique, écarts, KPI.

Worker IA associé (Python)

irrigation-ml-worker (Python / FastAPI / scikit-learn / XGBoost / PyTorch) : entraînement walk-forward, prédiction avec quantiles ; modèles versionnés via MLflow.

Exemples de contrats REST

```
POST /api/sensors/readings ; GET /api/weather/parcels/{id}
POST /api/irrigation/recommendations {parcelId, horizonDays}
POST /api/irrigation/events ; GET /api/irrigation/parcels/{id}/history
```

1.2.8 Risques et contrôles

Les datasets climatiques publics ne contiennent pas la « vraie » dose optimale par parcelle ; adaptation pédologique et supervision agronomique obligatoires.

1.2.9 Livrables

Baseline FAO-56, comparaison ML/hybride, API de recommandation sous contraintes, journal des irrigations et métriques terrain.

Sujet 1.3 — [OLV-6] Détection temporelle des oliviers dépérissants ou morts

Sujet également présent dans le Groupe 3 (fiche identique).

1.3.1 Problématique

Suivre un même arbre à plusieurs dates, quantifier les changements de couronne et de vigueur et identifier les individus nécessitant une inspection.

1.3.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
OQDS Insight (Data in Brief)	https://doi.org/10.1016/j.dib.2025.111615	Données de qualité/état d'oliviers ; référence pour le suivi.	Document initial
NEON tree crowns DTA (Hugging Face)	https://huggingface.co/datasets/weecology/neon-tree-crowns-dta	Couronnes d'arbres multi-dates (autres essences) ; transfert méthodologique.	Document initial
Olive hyperspectral stress (Zenodo)	https://zenodo.org/records/10336545	Signatures spectrales de stress.	Document initial
OliveTreeCrownsDb (Mendeley)	https://data.mendeley.com/datasets/xym8rd2srf/2	Référence initiale non temporelle pour les couronnes.	Document initial
Collecte Maroc	Données internes / à fournir	tree_id stable, orthomosaïques répétées, couronnes, date, indices de vigueur, statut confirmé sain/stressé/mort.	À constituer

1.3.3 Algorithmes à comparer

Approche	Modèle	Rôle de comparaison
Baseline	Seuil de variation NDVI/couronne	Approche explicable
Statistique	CUSUM ou détection de rupture	Détection chronologique
Anomalie	Isolation Forest	Variables de changement
Siamois	Siamese CNN deux dates	Similarité visuelle
Temps	Autoencoder temporel / TCN	Séquences nombreuses et alignées

1.3.4 Optimisation

- Optimiser en premier le recalage : points homologues, erreur géométrique, résolution commune, stabilité des tree_id.
- Siamese : contrastive/triplet loss, AdamW, learning rate, équilibrage des paires positives/négatives.
- Séries : longueur de fenêtre, seuil d'alerte, persistance temporelle et filtrage des changements saisonniers.
- Ruptures/statistiques : seuils et pénalité calibrés uniquement en validation ; Optuna TPE si l'espace est suffisant.
- Modèle léger ONNX pour l'inférence à l'échelle de l'arbre ; mesure du coût par 1 000 arbres.

1.3.5 Protocole expérimental

1. Constituer des couples et séquences d'observation des mêmes arbres avec co-enregistrement.
2. Contrôler l'identité des arbres entre saisons, les changements de couronne et les doublons possibles.
3. Comparer seuils experts, rupture statistique, Isolation Forest et Siamese si les exemples étiquetés suffisent.
4. Évaluer sur exploitations non vues et campagnes postérieures, sans paire partageant les mêmes arbres entre splits.

5. Mettre en place l'inspection agronomique et l'historique des décisions avant toute alerte définitive.

1.3.6 Indicateurs de performance

Rappel des arbres dépérissants confirmés, précision, délai avant validation terrain, alertes répétées, erreur de correspondance.

1.3.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
tree-tracking-service	Maintient l'identité (tree_id) de chaque arbre entre acquisitions : appariement spatial (distance, IoU des couronnes), gestion des fusions, scissions et doublons, matrice de correspondance entre dates.	Entrée : observations d'arbres. Sortie : tree_id stable, correspondances.
change-analysis-service	Recalage des orthomosaïques (points homologues), extraction des différences de couronne et de vigueur, calcul des indicateurs de changement via un worker Python.	Entrée : parcelle, dateFrom, dateTo. Sortie : indicateurs de changement par arbre.
decline-risk-service	Calcule le score de dépérissement par arbre, applique seuils, persistance temporelle et filtrage saisonnier, et tient la chronologie ; classe l'arbre en « candidat à inspecter ».	Entrée : indicateurs. Sortie : score, statut, chronologie.
field-inspection-service	Missions de terrain (liste d'arbres à inspecter, itinéraire), photos, confirmation sain/stressé/mort, historique des décisions et retour des labels au dataset-service.	Entrée : candidats. Sortie : confirmation, photos, labels.

Worker IA associé (Python)

change-detection-worker (Python / FastAPI / PyTorch / OpenCV / ruptures) : recalage, Siamese CNN, détection de rupture, export ONNX.

Exemples de contrats REST

```
POST /api/trees/{id}/observations
POST /api/tree-changes/jobs {parcelId, dateFrom, dateTo}
GET /api/trees/{id}/health-history ; POST /api/inspections/{id}/confirm
```

1.3.8 Risques et contrôles

Jeux externes insuffisants pour apprendre tous les scénarios marocains ; recalage incorrect = fausse alerte ; nécessité d'observations répétées.

1.3.9 Livrables

Suivi multi-dates d'arbres individuels, comparaison de 3–4 algorithmes, carte des évolutions et workflow d'inspection.

Sujet 1.4 — [SAI-1] AquaSmart — Prévision intelligente des ressources hydriques

Sujet également présent dans le Groupe 2 (fiche identique).

1.4.1 Problématique

Construire une plateforme de surveillance de niveaux ou de consommation d'eau et de prévision à court terme, sans modélisation d'équations différentielles.

1.4.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
Jena Climate (exemple Keras)	https://keras.io/examples/timeseries/timeseries_weather_forecasting/	Série météo ouverte utilisée comme proxy ; ne représente pas des volumes hydriques réels.	Document initial
Jena Climate — source Max Planck	https://www.bgc-jena.mpg.de/wetter/	Source d'origine de la série météo (téléchargement et documentation).	Ajout
USGS Water Data	https://waterdata.usgs.gov/	Séries réelles de niveaux/débits pour une application réellement hydrique (alternative).	Ajout (à vérifier)
Dataset de consommations (enseignant)	Données internes / à fournir	Pour une application réellement hydrique : jeu fourni et documenté par l'enseignant.	À fournir

Code de départ

Ressource	Lien	Usage	Origine
Code de départ : TCN (locuslab)	https://github.com/locuslab/TCN	Implémentation de référence du TCN (PyTorch).	Document initial
keras-tcn	https://github.com/philipperemy/keras-tcn	Alternative Keras pour le TCN.	Ajout

1.4.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	LSTM	Dépendances temporelles longues, fenêtre glissante
Avancée (imposée)	TCN léger	Convolutions causales et dilatées, entraînement parallèle

Concepts à comprendre : Fenêtre temporelle ; dépendance dans le temps ; convolutions causales/dilatées.

1.4.4 Optimisation

- Prévision un pas ou plusieurs pas ; fenêtres de 24 à 72 pas.
- Entraînement des deux modèles sur la même cible.
- Tuning de 2 à 3 paramètres (fenêtre, learning rate, dilatation/taille de noyau).

1.4.5 Protocole expérimental

1. Découpage chronologique train/validation/test, sans mélange aléatoire.
2. Normalisation ajustée sur le train seulement.
3. Mêmes partitions et même cible pour LSTM et TCN.
4. Archiver métriques et courbes d'apprentissage.

1.4.6 Indicateurs de performance

MAE, RMSE, temps d'inférence.

1.4.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
measurement-service	Importe (CSV/JSON) et expose les séries de mesures ; valide les colonnes, les pas de temps et les valeurs manquantes ; stocke dans PostgreSQL.	Entrée : fichier de mesures. Sortie : observations, identifiants de série.
forecast-service	Construit la fenêtre d'entrée, appelle ai-inference, enregistre la prévision (horizon, modèle, version) et la restitue.	Entrée : série, horizon, modèle. Sortie : prévision archivée.
alert-service	Compare mesures et prévisions à des seuils configurables, enregistre et notifie les alertes.	Entrée : valeurs prévues, seuils. Sortie : alertes historisées.
ai-inference (Python / FastAPI)	Charge les modèles LSTM et TCN entraînés, applique la même normalisation que l'entraînement et renvoie la prévision.	Entrée : fenêtre normalisée. Sortie : valeurs prédites, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

1.4.8 Limites et cadrage de faisabilité

MVP avec une seule série multivariée et un horizon fixe ; données hydriques réelles seulement si accessibles ; Jena Climate n'est pas une donnée hydrique.

1.4.9 Scénario de démonstration et livrables

Scénario : Importer des mesures, lancer la prévision, afficher réel/prédit et enregistrer une alerte.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Sujet 1.5 — [SAI-3] MedVision Smart — Segmentation biomédicale intelligente

Sujet également présent dans le Groupe 3 (fiche identique).

1.5.1 Problématique

Développer une plateforme de segmentation d'images avec entraînement, comparaison et contrôle humain des masques.

1.5.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
BUSI — Breast Ultrasound Images (Kaggle)	https://www.kaggle.com/datasets/aryashah2k/breast-ultrasound-images-dataset	À utiliser si masques et licence sont validés.	Document initial
Kvasir-SEG	https://datasets.simula.no/kvasir-seg/	Alternative : segmentation binaire à masques vérifiés.	Ajout (à vérifier)

Code de départ

Ressource	Lien	Usage	Origine
segmentation_models.pytorch	https://github.com/qubvel-org/segmentation_models.pytorch	U-Net et U-Net++ avec encodeurs pré-entraînés.	Document initial
UNet++ (référence originale)	https://github.com/MrGiovanni/UNetPlusPlus	Implémentation de l'article d'origine.	Ajout

1.5.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	U-Net	Segmentation binaire par encodeur-décodeur
Avancée (imposée)	U-Net++	Skip connections denses imbriquées

Concepts à comprendre : Encodeur-décodeur, skip connections, masque de segmentation, Dice.

1.5.4 Optimisation

- Entraîner U-Net et U-Net++ sur images 128–256 px.
- Tester deux fonctions de perte (ex. BCE+Dice vs Dice) ou deux jeux d'hyperparamètres.

1.5.5 Protocole expérimental

1. Split par patient/image, sans fuite entre train et test.
2. Mêmes partitions et même prétraitement pour les deux modèles.
3. Analyse d'échecs : cas où les masques divergent.
4. Validation humaine d'une sortie dans l'interface.

1.5.6 Indicateurs de performance

Dice, IoU, rappel, coût d'inférence ; analyse d'échecs.

1.5.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
exam-image-service	Gère images et métadonnées fictives (aucune donnée identifiante) ; stockage et consultation.	Entrée : image + métadonnées. Sortie : examId, URI.

Microservice	Description et responsabilité	Entrées → sorties
segmentation-service	Lance les tâches d'inférence pour U-Net et U-Net++, enregistre les masques et métriques.	Entrée : examId, modèle. Sortie : masques par modèle.
mask-review-service	Visualise et compare les masques, permet corrections et validation par un humain, avec historique.	Entrée : masque, décision. Sortie : masque validé/corrigé.
ai-inference (Python / FastAPI)	Charge U-Net et U-Net++, prétraite l'image et renvoie les masques binaires.	Entrée : image. Sortie : masque, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

1.5.8 Limites et cadrage de faisabilité

Jeu de données réduit ; pas de données identifiantes ; résultats à visée pédagogique uniquement.

1.5.9 Scénario de démonstration et livrables

Scénario : Charger une image, obtenir deux masques, comparer et valider une sortie.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Sujet 1.6 — [SAI-4] EcoSense Smart — Analyse intelligente de mesures environnementales

Sujet également présent dans le Groupe 2 (fiche identique).

1.6.1 Problématique

Prédire l'évolution de mesures environnementales et signaler des valeurs atypiques à partir de séries multivariées.

1.6.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
Jena Climate (exemple Keras)	https://keras.io/examples/timeseries/timeseries_weather_forecasting/	Modèle entraîné sur quelques variables sélectionnées.	Document initial
UCI Air Quality	https://archive.ics.uci.edu/dataset/360/air+quality	Si l'on veut des polluants réels (CO, NOx, benzène).	Ajout (à vérifier)

Code de départ

Ressource	Lien	Usage	Origine
Transformer pour séries temporelles (Keras)	https://keras.io/examples/timeseries/timeseries_classification_transformer/	Modèle de classification à adapter à la régression.	Document initial

1.6.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	GRU	Séquences avec mémoire légère
Avancée (imposée)	Transformer temporel compact	Attention, un bloc, peu de têtes

Concepts à comprendre : Séquences, attention, embeddings temporels, normalisation ; adapter le modèle de classification fourni à la régression.

1.6.4 Optimisation

- Deux modèles entraînés sur la même tâche de prévision.
- Un bloc d'attention, peu de têtes.
- Étudier l'erreur selon l'horizon.

1.6.5 Protocole expérimental

1. Découpage chronologique.
2. Mêmes variables, fenêtres et horizons pour GRU et Transformer.
3. Erreur par variable et par horizon.
4. Règle d'alerte validée sur valeurs atypiques simulées ou réelles.

1.6.6 Indicateurs de performance

MAE, RMSE, erreur par variable, latence.

1.6.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
observation-service	Ingestion et consultation des mesures multi-capteurs ; contrôle des plages et des valeurs manquantes.	Entrée : mesures. Sortie : observations, tendances.
prediction-service	Appelle ai-inference et archive les prévisions par variable et par horizon.	Entrée : capteur, horizon, modèle. Sortie : prévision archivée.
environment-alert-service	Applique des règles d'alerte (dépassement de seuil, valeur atypique) et historise.	Entrée : mesures/prévisions. Sortie : alertes.
ai-inference (Python / FastAPI)	Charge GRU et Transformer, renvoie la prévision multivariée.	Entrée : fenêtre normalisée. Sortie : valeurs prédites, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

1.6.8 Limites et cadrage de faisabilité

Ne pas annoncer la prédiction de pollution si le dataset choisi ne contient pas de polluants (Jena Climate n'en contient pas).

1.6.9 Scénario de démonstration et livrables

Scénario : Afficher les capteurs et leurs tendances, prévoir des valeurs et détecter un dépassement.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Sujet 1.7 — [SAI-5] IndustryGuard Smart — Surveillance industrielle par séries temporelles

Sujet également présent dans le Groupe 2 (fiche identique).

1.7.1 Problématique

Classer des séries issues de capteurs ou de vibrations et signaler des comportements anormaux.

1.7.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
FordA (exemple Keras)	https://keras.io/examples/timeseries/timeseries_classification_from_scratch/	Séries de mesures de moteurs ; classification, pas durée de vie restante.	Document initial
FordA — UCR/UEA archive	https://www.timeseriesclassification.com/description.php?Dataset=FordA	Source et description officielles du jeu.	Ajout
CWRU Bearing Data Center	https://engineering.case.edu/bearingdatacenter	Option : vibrations de roulements (défauts) pour enrichir le sujet.	Ajout (à vérifier)

Code de départ

Ressource	Lien	Usage	Origine
TCN (locuslab)	https://github.com/locuslab/TCN	Implémentation de référence du TCN.	Document initial

1.7.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	CNN 1D	Motifs locaux dans le signal
Avancée (imposée)	TCN	Convolutions causales dilatées, large champ réceptif

Concepts à comprendre : Convolutions 1D, séries temporelles, noyaux dilatés, classification.

1.7.4 Optimisation

- Entraîner CNN 1D et TCN sur les mêmes partitions.
- Tester la longueur de séquence et le dropout.

1.7.5 Protocole expérimental

1. Partitions identiques pour les deux modèles.
2. Respecter le split officiel train/test de FordA.
3. Matrice de confusion et rappel par classe.
4. Historiser chaque alerte avec score.

1.7.6 Indicateurs de performance

F1 macro, rappel, matrice de confusion, latence.

1.7.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
equipment-service	Inventaire simulé des équipements et association aux signaux.	Entrée : équipement. Sortie : equipmentId, métadonnées.
telemetry-service	Import et stockage des signaux (séquences) ; consultation par équipement.	Entrée : séquence de mesures. Sortie : signalId.
anomaly-service	Demande la classification à ai-inference, enregistre classe/score et génère l'alerte si anomalie.	Entrée : signalId, modèle. Sortie : classe, score, alerte.
ai-inference (Python / FastAPI)	Charge CNN 1D et TCN, renvoie classe et probabilité.	Entrée : séquence normalisée. Sortie : classe, score, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

1.7.8 Limites et cadrage de faisabilité

Ne pas présenter une classification FordA comme une prédiction du temps avant panne.

1.7.9 Scénario de démonstration et livrables

Scénario : Charger une séquence, obtenir classe/score et historiser l'alerte.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Sujet 1.8 — [SAI-6] EnergyTwin Smart — Prévision intelligente de consommation électrique

Sujet également présent dans le Groupe 2 (fiche identique).

1.8.1 Problématique

Prévoir la consommation d'un bâtiment avec un service d'expérimentation et des alertes de dépassement.

1.8.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
UCI Appliances Energy Prediction	https://archive.ics.uci.edu/dataset/374/appliances+energy+prediction	19 735 observations, 28 variables, pas de 10 minutes.	Document initial
UCI Household Electric Power Consumption	https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption	Alternative/complément : consommation d'un foyer à la minute.	Ajout (à vérifier)

Code de départ

Ressource	Lien	Usage	Origine
Prévision météo (Keras)	https://keras.io/examples/timeseries/timeseries_weather_forecasting/	Pipeline LSTM de départ.	Document initial
Transformer séries temporelles (Keras)	https://keras.io/examples/timeseries/timeseries_classification_transformer/	Exemple de bloc d'attention.	Document initial
Couche Attention (Keras)	https://keras.io/api/layers/attention_layers/attention/	Documentation de la couche à intégrer au LSTM.	Ajout

1.8.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	LSTM	Prévision supervisée sur séquences
Avancée (imposée)	LSTM avec couche d'attention	Pondération des pas de temps pertinents

Concepts à comprendre : Prévision supervisée, séquences, couche attention, attribution de poids (sans interprétation causale).

1.8.4 Optimisation

- Comparer LSTM simple à LSTM + attention (couches Keras).
- 2 à 3 configurations (unités, fenêtre, learning rate).

1.8.5 Protocole expérimental

1. Découpage chronologique.
2. Mêmes variables et fenêtres pour les deux modèles.
3. Erreur par créneau horaire.
4. Seuil d'alerte défini sur validation.

1.8.6 Indicateurs de performance

MAE, RMSE, erreur selon créneau horaire, latence.

1.8.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
meter-service	Import CSV et consultation des séries de consommation ; contrôle de cohérence.	Entrée : CSV. Sortie : séries, historique.
energy-forecast-service	Appelle ai-inference, archive les prévisions et compare LSTM / LSTM-attention.	Entrée : fenêtre, modèle. Sortie : prévision archivée.
threshold-service	Gère les seuils de dépassement et enregistre les alertes.	Entrée : seuil, prévisions. Sortie : alertes.
ai-inference (Python / FastAPI)	Charge les deux modèles, renvoie prévision et, si disponible, poids d'attention.	Entrée : fenêtre normalisée. Sortie : prévision, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

1.8.8 Limites et cadrage de faisabilité

Un bâtiment, pas de véritable réseau électrique ; prédiction plutôt qu'optimisation du réseau.

1.8.9 Scénario de démonstration et livrables

Scénario : Importer un CSV, visualiser historique/prévisions et créer un seuil d'alerte.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Groupe 2 — Séries temporelles et prévision

Sujets de prévision et de classification sur séries (météo, eau, énergie, trafic, signaux industriels, rendement).

Code	Sujet
OLV-3	2.1 — Estimation intelligente des besoins en irrigation
OLV-4	2.2 — Analyse satellitaire multi-temporelle des oliveraies
OLV-5	2.3 — Prévision du rendement des oliveraies
SAI-1	2.4 — AquaSmart — Prévision intelligente des ressources hydriques
SAI-4	2.5 — EcoSense Smart — Analyse intelligente de mesures environnementales
SAI-5	2.6 — IndustryGuard Smart — Surveillance industrielle par séries temporelles
SAI-6	2.7 — EnergyTwin Smart — Prévision intelligente de consommation électrique
SAI-7	2.8 — UrbanFlow Smart — Prévision du trafic par réseau de graphes

Sujet 2.1 — [OLV-3] Estimation intelligente des besoins en irrigation

Sujet également présent dans le Groupe 1 (fiche identique).

2.1.1 Problématique

Prédire un besoin hydrique conseillé à horizon court en respectant la sécurité agronomique, et comparer les recommandations à une méthode FAO-56 de référence.

2.1.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
NASA POWER	https://power.larc.nasa.gov/	Météo journalière (rayonnement, température, vent, humidité, pluie).	Document initial
ERA5-Land (Earth Engine)	https://developers.google.com/earth-engine/datasets/catalog/ECMWF_ERA5_LAND_DAILY_AGGR	Réanalyse climatique, humidité du sol, évapotranspiration.	Document initial
FAO WaPOR	https://www.fao.org/in-action/remote-sensing-for-water-productivity/wapor-data-access/en	Évapotranspiration et productivité de l'eau par télédétection.	Document initial
FAO-56 (Irrigation and Drainage Paper 56)	https://www.fao.org/4/x0490e/x0490e00.htm	Référence méthodologique pour ET0 (Penman-Monteith), Kc et bilan hydrique.	Ajout
SoilGrids (ISRIC)	https://soilgrids.org/	Propriétés des sols (texture, matière organique) pour la réserve utile.	Ajout
Open-Meteo	https://open-meteo.com/	Prévisions météo à 3–14 jours pour les horizons de recommandation.	Ajout
Collecte Maroc (indispensable)	Données internes / à fournir	ET0 et Kc contextualisés, pluies, sol, humidité de sondes, volumes réellement appliqués, rendement, disponibilité de l'eau.	À constituer

2.1.3 Algorithmes à comparer

Approche	Modèle	Rôle de comparaison
Référence physique	FAO-56 : ETc = Kc × ET0 + bilan hydrique	Baseline agronomique
Tabulaire	Random Forest	Robustesse simple
Tabulaire	XGBoost / CatBoost	Relations non linéaires
Temporel	LSTM / TCN	Séquences météo et humidité
Hybride	FAO-56 + XGBoost résiduel	Correction physique pilotée par les données

2.1.4 Optimisation

- RF/XGBoost : profondeur, n_estimators, learning_rate, min_child_weight, subsample ; pas d'AdamW pour les arbres de décision.
- LSTM/TCN : Adam ou AdamW, 1e-4 à 1e-3, fenêtres de 7/14/30 jours, dropout, hidden size, early stopping.
- Hybride : optimiser l'erreur résiduelle à FAO-56 ; imposer des bornes d'eau et des règles métier explicites.

- Optuna TPE ou random search avec validation temporelle en chaîne (walk-forward) ; comparaison à coût de calcul équivalent.
- Tester les horizons 3, 7 et 14 jours ; quantiles ou intervalles conformes pour l'incertitude.

2.1.5 Protocole expérimental

1. Constituer des séries par parcelle, dater les interventions, ne pas remplir artificiellement les données critiques manquantes.
2. Entraîner sur les campagnes anciennes, valider sur la période récente, tester sur la campagne suivante.
3. Comparer au minimum FAO-56, RF, XGBoost et hybride ; LSTM si les données le permettent.
4. Évaluer hors ligne, puis sur parcelles pilotes avec groupe témoin si les partenaires agronomiques l'autorisent.
5. Bloquer les recommandations lorsque sondes ou prévisions sont périmées ou incohérentes.

2.1.6 Indicateurs de performance

MAE/RMSE (volume), erreur d'humidité, m³/ha économisés, rendement, proportion de recommandations risquées.

2.1.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
weather-data-service	Collecte NASA POWER, ERA5-Land et prévisions, normalise les unités, le pas de temps et le fuseau, gère le cache et la qualité des séries, et expose observations et prévisions par parcelle.	Entrée : parcelle, période. Sortie : séries météo normalisées.
soil-sensor-service	Ingestion IoT (HTTP/MQTT) des sondes d'humidité, calibration, détection des sondes périmées ou incohérentes, séries d'humidité par profondeur et par parcelle.	Entrée : mesures capteurs. Sortie : séries d'humidité, indicateur de fiabilité.
water-balance-service	Calcule ET0 (Penman-Monteith FAO-56), ETc = Kc × ET0 et le bilan hydrique (pluie efficace, réserve utile, drainage) par parcelle ; fournit la baseline physique.	Entrée : météo, sol, Kc. Sortie : ET0, ETc, déficit hydrique.
irrigation-recommendation-service	Appelle le worker (RF, XGBoost, LSTM ou hybride), applique les règles de sécurité (bornes, blocage si données périmées), produit dose, planning et intervalle d'incertitude à 3/7/14 jours.	Entrée : parcelle, horizon. Sortie : recommandation, planning, incertitude.
irrigation-tracking-service	Enregistre les volumes réellement appliqués, compare recommandation et réalité, calcule les m³ économisés et alimente le réentraînement.	Entrée : événements d'irrigation. Sortie : historique, écarts, KPI.

Worker IA associé (Python)

irrigation-ml-worker (Python / FastAPI / scikit-learn / XGBoost / PyTorch) : entraînement walk-forward, prédiction avec quantiles ; modèles versionnés via MLflow.

Exemples de contrats REST

```
POST /api/sensors/readings ; GET /api/weather/parcels/{id}
POST /api/irrigation/recommendations {parcelId, horizonDays}
POST /api/irrigation/events ; GET /api/irrigation/parcels/{id}/history
```

2.1.8 Risques et contrôles

Les datasets climatiques publics ne contiennent pas la « vraie » dose optimale par parcelle ; adaptation pédologique et supervision agronomique obligatoires.

2.1.9 Livrables

Baseline FAO-56, comparaison ML/hybride, API de recommandation sous contraintes, journal des irrigations et métriques terrain.

Sujet 2.2 — [OLV-4] Analyse satellitaire multi-temporelle des oliveraies

Sujet également présent dans le Groupe 3 (fiche identique).

2.2.1 Problématique

Détecter des ruptures ou anomalies de vigueur à l'échelle de la parcelle sur des séries optiques multi-dates, en tenant compte des nuages, des saisons et des pratiques culturales.

2.2.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
Sentinel-2 L2A (Earth Engine)	https://developers.google.com/earth-engine/datasets/catalog/COPERNICUS_S2_SR_HARMONIZED	Séries NDVI/NDRE/NDWI/SAVI à 10–20 m, masques QA.	Document initial
Landsat Collection 2 (USGS)	https://www.usgs.gov/landsat-missions/landsat-collection-2	Historique long (30 m) pour la baseline saisonnière.	Document initial
ERA5-Land (météo)	https://developers.google.com/earth-engine/datasets/catalog/ECMWF_ERA5_LAND_DAILY_AGGR	Variables météo pour la fusion et l'ablation.	Document initial
Copernicus Data Space Ecosystem	https://dataspace.copernicus.eu/	Accès alternatif aux scènes Sentinel hors Earth Engine.	Ajout
Collecte Maroc	Données internes / à fournir	Polygones géoréférencés, visites terrain datées, irrigation, observations de stress ; masques nuage et qualité satellite.	À constituer

2.2.3 Algorithmes à comparer

Approche	Modèle	Rôle de comparaison
Règles expertes	Seuil z-score saisonnier NDVI/NDRE	Baseline transparente
Non supervisé	Isolation Forest	Anomalies multivariées
Non supervisé	One-Class SVM	Frontière de normalité
Reconstruction	Autoencoder / LSTM-AE	Écarts à la saisonnalité
Supervisé	XGBoost + météo	Si anomalies terrain étiquetées

2.2.4 Optimisation

- Agrégation temporelle 5–15 jours, pourcentage de nuage maximal et filtres de qualité ; attention aux pixels mixtes.
- Isolation Forest : n_estimators, max_samples, contamination (réglée sans utiliser le test). One-Class SVM : gamma et nu.
- Autoencoder : AdamW, lr, latent_dim, longueur de fenêtre, reconstruction loss.
- Optimisation bayésienne/TPE sur validation temporelle et campagnes distinctes, avec budget d'essais contrôlé.
- Ablation : NDVI seul vs NDVI+NDRE+NDWI+SAVI vs fusion météo ; lisser sans masquer les alertes réellement brusques.

2.2.5 Protocole expérimental

1. Extraire les images via Earth Engine, appliquer les masques QA, fixer période et couverture par parcelle.
2. Calculer les séries NDVI/NDRE/NDWI/SAVI puis normaliser par saison et par parcelle.
3. Comparer règles expertes et 2–4 méthodes statistiques/ML ; mesurer la stabilité interannuelle.
4. Faire correspondre alertes et observations terrain avec une tolérance temporelle prédéfinie.
5. Tester un bassin ou une campagne jamais vue pour la robustesse spatiale et temporelle.

2.2.6 Indicateurs de performance

Précision, rappel, F1 des alertes, délai moyen de détection, faux positifs par parcelle, stabilité interannuelle.

2.2.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
satellite-acquisition-service	Orchestre l'extraction Earth Engine / Copernicus, sélectionne les scènes par parcelle et période, applique les masques nuage/QA et tient un catalogue de scènes avec leur couverture utile.	Entrée : import-job (parcelle, dates). Sortie : scènes filtrées, catalogue.
vegetation-index-service	Calcule NDVI, NDRE, NDWI, SAVI par parcelle (statistiques zonales), agrège par fenêtre de 5–15 jours, normalise par saison et archive les séries.	Entrée : scènes. Sortie : séries d'indices par parcelle.
anomaly-detection-service	Lance les détecteurs (z-score, Isolation Forest, One-Class SVM, autoencoder) via un worker Python et renvoie des anomalies datées avec score et variable explicative.	Entrée : séries d'indices. Sortie : anomalies datées, scores, interprétation.
parcel-monitoring-service	Cartes temporelles, courbes de vigueur, saisie des validations terrain, appariement alerte/observation avec tolérance temporelle et calcul des KPI (faux positifs, délai de détection).	Entrée : anomalies, observations terrain. Sortie : tableau de bord, KPI.

Worker IA associé (Python)

satellite-analytics-worker (Python / FastAPI / scikit-learn / PyTorch, rasterio, Earth Engine API) : calcul d'indices, détection d'anomalies, entraînement des autoencodeurs.

Exemples de contrats REST

```
POST /api/satellite/import-jobs {parcelId, startDate, endDate}
GET /api/parcels/{id}/vegetation-series
POST /api/anomalies/analyze ; GET /api/parcels/{id}/anomalies
```

2.2.8 Risques et contrôles

Nuages résiduels ; résolution satellite insuffisante pour les arbres individuels ; fausses ruptures dues à la phénologie.

2.2.9 Livrables

Pipeline géospatial reproductible, modèles comparés, couches cartographiques et tableau de bord d'alertes datées.

Sujet 2.3 — [OLV-5] Prévision du rendement des oliveraies

2.3.1 Problématique

Prédire le rendement au niveau parcelle/campagne avant la récolte, à partir des historiques, du climat, des signaux satellites et éventuellement du nombre d'arbres.

2.3.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
FAOSTAT (QCL)	https://www.fao.org/faostat/en/#data/QCL	Contexte pays et séries historiques de production d'olives.	Document initial
NASA POWER	https://power.larc.nasa.gov/	Variables climatiques de la campagne.	Document initial
Sentinel-2 (Earth Engine)	https://developers.google.com/earth-engine/datasets/catalog/COPERNICUS_S2_SR_HARMONIZED	Indices de végétation avant récolte.	Document initial
FAO WaPOR	https://www.fao.org/in-action/remote-sensing-for-water-productivity/wapor-data-access/en	Évapotranspiration et productivité de l'eau.	Document initial
Collecte Maroc (obligatoire)	Données internes / à fournir	Récolte réelle kg/ha, variété, âge, alternance, surfaces, arbres, irrigation, maladies, dates et campagne.	À constituer

2.3.3 Algorithmes à comparer

Approche	Modèle	Rôle de comparaison
Baseline	Moyenne des campagnes + régression ridge	Référence minimale
Ensemble	Random Forest	Non-linéarité tabulaire
Boosting	XGBoost / CatBoost	Modèle principal potentiel
Réseau	MLP	Comparaison réseau de neurones
Temps	Temporal Fusion / LSTM léger	Seulement si séries suffisantes

2.3.4 Optimisation

- Boosting : Optuna sur depth, learning_rate, estimators, subsample et régularisation ; early stopping sur validation.
- MLP/LSTM : AdamW, taux d'apprentissage, batch size, dropout et weight decay.
- Comparer les features disponibles 6 mois, 3 mois et 1 mois avant récolte pour éviter toute fuite de données futures.
- Intervalles de prédiction : quantile regression et conformal prediction avec calibration temporelle.
- Ablation : historique seul vs météo vs météo+satellite vs ajout des observations drone ; mesurer le gain réel.

2.3.5 Protocole expérimental

1. Définir une ligne = parcelle × campagne ; figer à l'avance la date de prévision.
2. Validation chronologique par campagne et test inter-région distinct ; contrôler les dépendances entre parcelles.
3. Comparer baseline/ridge/RF/XGBoost/CatBoost et mesurer le biais par variété et par mode d'irrigation.
4. Évaluer avec et sans données de récolte passée pour mesurer la dépendance à l'historique.
5. Présenter estimation, intervalle et variables explicatives, sans annoncer de garantie de rendement.

2.3.6 Indicateurs de performance

MAE et RMSE (kg/ha), erreur relative robuste, R^2, biais par région, couverture des intervalles.

2.3.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
harvest-record-service	Déclaration des récoltes (kg, surface, date, campagne), contrôles de cohérence (rendement kg/ha plausible), validation et historique par parcelle.	Entrée : déclaration de récolte. Sortie : récolte validée.
yield-feature-service	Construit la table parcelle × campagne : jointure temporelle du climat, des indices de végétation, de l'inventaire d'arbres, de l'irrigation et des maladies. Respecte forecast_date pour interdire les fuites ; versionne les jeux de features.	Entrée : parcelle, campagne, forecastDate. Sortie : vecteur de features versionné.
yield-forecast-service	Orchestre le worker (ridge, RF, XGBoost, CatBoost, MLP), renvoie estimation, intervalle (quantile/conformal) et explications (importance des variables, SHAP), avec model_version.	Entrée : features. Sortie : prévision, intervalle, explication.
harvest-planning-service	Scénarios (prudent / médian / optimiste), besoins en main-d'œuvre et logistique, suivi prévision vs réalisation et biais par région.	Entrée : prévisions. Sortie : plan de récolte, écarts.

Worker IA associé (Python)

yield-ml-worker (Python / FastAPI / scikit-learn / XGBoost / CatBoost / PyTorch) : entraînement chronologique, calibration des intervalles, explicabilité.

Exemples de contrats REST

```
POST /api/harvests ; GET /api/parcels/{id}/harvests
POST /api/yield-forecasts {parcelId, campaign, forecastDate}
GET /api/yield-forecasts/{id}
```

2.3.8 Risques et contrôles

FAOSTAT ne remplace pas les rendements par parcelle ; risque élevé de fuite d'information si des variables de fin de campagne sont utilisées trop tôt.

2.3.9 Livrables

Dataset parcelle-campagne local, 3–5 régressions comparées, intervalles prédictifs, API et tableau de planification.

Sujet 2.4 — [SAI-1] AquaSmart — Prévision intelligente des ressources hydriques

Sujet également présent dans le Groupe 1 (fiche identique).

2.4.1 Problématique

Construire une plateforme de surveillance de niveaux ou de consommation d'eau et de prévision à court terme, sans modélisation d'équations différentielles.

2.4.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
Jena Climate (exemple Keras)	https://keras.io/examples/timeseries/timeseries_weather_forecasting/	Série météo ouverte utilisée comme proxy ; ne représente pas des volumes hydriques réels.	Document initial
Jena Climate — source Max Planck	https://www.bgc-jena.mpg.de/wetter/	Source d'origine de la série météo (téléchargement et documentation).	Ajout
USGS Water Data	https://waterdata.usgs.gov/	Séries réelles de niveaux/débits pour une application réellement hydrique (alternative).	Ajout (à vérifier)
Dataset de consommations (enseignant)	Données internes / à fournir	Pour une application réellement hydrique : jeu fourni et documenté par l'enseignant.	À fournir

Code de départ

Ressource	Lien	Usage	Origine
Code de départ : TCN (locuslab)	https://github.com/locuslab/TCN	Implémentation de référence du TCN (PyTorch).	Document initial
keras-tcn	https://github.com/philipperemy/keras-tcn	Alternative Keras pour le TCN.	Ajout

2.4.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	LSTM	Dépendances temporelles longues, fenêtre glissante
Avancée (imposée)	TCN léger	Convolutions causales et dilatées, entraînement parallèle

Concepts à comprendre : Fenêtre temporelle ; dépendance dans le temps ; convolutions causales/dilatées.

2.4.4 Optimisation

- Prévision un pas ou plusieurs pas ; fenêtres de 24 à 72 pas.
- Entraînement des deux modèles sur la même cible.
- Tuning de 2 à 3 paramètres (fenêtre, learning rate, dilatation/taille de noyau).

2.4.5 Protocole expérimental

1. Découpage chronologique train/validation/test, sans mélange aléatoire.
2. Normalisation ajustée sur le train seulement.
3. Mêmes partitions et même cible pour LSTM et TCN.
4. Archiver métriques et courbes d'apprentissage.

2.4.6 Indicateurs de performance

MAE, RMSE, temps d'inférence.

2.4.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
measurement-service	Importe (CSV/JSON) et expose les séries de mesures ; valide les colonnes, les pas de temps et les valeurs manquantes ; stocke dans PostgreSQL.	Entrée : fichier de mesures. Sortie : observations, identifiants de série.
forecast-service	Construit la fenêtre d'entrée, appelle ai-inference, enregistre la prévision (horizon, modèle, version) et la restitue.	Entrée : série, horizon, modèle. Sortie : prévision archivée.
alert-service	Compare mesures et prévisions à des seuils configurables, enregistre et notifie les alertes.	Entrée : valeurs prévues, seuils. Sortie : alertes historisées.
ai-inference (Python / FastAPI)	Charge les modèles LSTM et TCN entraînés, applique la même normalisation que l'entraînement et renvoie la prévision.	Entrée : fenêtre normalisée. Sortie : valeurs prédites, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

2.4.8 Limites et cadrage de faisabilité

MVP avec une seule série multivariée et un horizon fixe ; données hydriques réelles seulement si accessibles ; Jena Climate n'est pas une donnée hydrique.

2.4.9 Scénario de démonstration et livrables

Scénario : Importer des mesures, lancer la prévision, afficher réel/prédit et enregistrer une alerte.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Sujet 2.5 — [SAI-4] EcoSense Smart — Analyse intelligente de mesures environnementales

Sujet également présent dans le Groupe 1 (fiche identique).

2.5.1 Problématique

Prédire l'évolution de mesures environnementales et signaler des valeurs atypiques à partir de séries multivariées.

2.5.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
Jena Climate (exemple Keras)	https://keras.io/examples/timeseries/timeseries_weather_forecasting/	Modèle entraîné sur quelques variables sélectionnées.	Document initial
UCI Air Quality	https://archive.ics.uci.edu/dataset/360/air+quality	Si l'on veut des polluants réels (CO, NOx, benzène).	Ajout (à vérifier)

Code de départ

Ressource	Lien	Usage	Origine
Transformer pour séries temporelles (Keras)	https://keras.io/examples/timeseries/timeseries_classification_transformer/	Modèle de classification à adapter à la régression.	Document initial

2.5.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	GRU	Séquences avec mémoire légère
Avancée (imposée)	Transformer temporel compact	Attention, un bloc, peu de têtes

Concepts à comprendre : Séquences, attention, embeddings temporels, normalisation ; adapter le modèle de classification fourni à la régression.

2.5.4 Optimisation

- Deux modèles entraînés sur la même tâche de prévision.
- Un bloc d'attention, peu de têtes.
- Étudier l'erreur selon l'horizon.

2.5.5 Protocole expérimental

1. Découpage chronologique.
2. Mêmes variables, fenêtres et horizons pour GRU et Transformer.
3. Erreur par variable et par horizon.
4. Règle d'alerte validée sur valeurs atypiques simulées ou réelles.

2.5.6 Indicateurs de performance

MAE, RMSE, erreur par variable, latence.

2.5.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
observation-service	Ingestion et consultation des mesures multi-capteurs ; contrôle des plages et des valeurs manquantes.	Entrée : mesures. Sortie : observations, tendances.
prediction-service	Appelle ai-inference et archive les prévisions par variable et par horizon.	Entrée : capteur, horizon, modèle. Sortie : prévision archivée.
environment-alert-service	Applique des règles d'alerte (dépassement de seuil, valeur atypique) et historise.	Entrée : mesures/prévisions. Sortie : alertes.
ai-inference (Python / FastAPI)	Charge GRU et Transformer, renvoie la prévision multivariée.	Entrée : fenêtre normalisée. Sortie : valeurs prédites, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

2.5.8 Limites et cadrage de faisabilité

Ne pas annoncer la prédiction de pollution si le dataset choisi ne contient pas de polluants (Jena Climate n'en contient pas).

2.5.9 Scénario de démonstration et livrables

Scénario : Afficher les capteurs et leurs tendances, prévoir des valeurs et détecter un dépassement.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Sujet 2.6 — [SAI-5] IndustryGuard Smart — Surveillance industrielle par séries temporelles

Sujet également présent dans le Groupe 1 (fiche identique).

2.6.1 Problématique

Classer des séries issues de capteurs ou de vibrations et signaler des comportements anormaux.

2.6.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
FordA (exemple Keras)	https://keras.io/examples/timeseries/timeseries_classification_from_scratch/	Séries de mesures de moteurs ; classification, pas durée de vie restante.	Document initial
FordA — UCR/UEA archive	https://www.timeseriesclassification.com/description.php?Dataset=FordA	Source et description officielles du jeu.	Ajout
CWRU Bearing Data Center	https://engineering.case.edu/bearingdatacenter	Option : vibrations de roulements (défauts) pour enrichir le sujet.	Ajout (à vérifier)

Code de départ

Ressource	Lien	Usage	Origine
TCN (locuslab)	https://github.com/locuslab/TCN	Implémentation de référence du TCN.	Document initial

2.6.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	CNN 1D	Motifs locaux dans le signal
Avancée (imposée)	TCN	Convolutions causales dilatées, large champ réceptif

Concepts à comprendre : Convolutions 1D, séries temporelles, noyaux dilatés, classification.

2.6.4 Optimisation

- Entraîner CNN 1D et TCN sur les mêmes partitions.
- Tester la longueur de séquence et le dropout.

2.6.5 Protocole expérimental

1. Partitions identiques pour les deux modèles.
2. Respecter le split officiel train/test de FordA.
3. Matrice de confusion et rappel par classe.
4. Historiser chaque alerte avec score.

2.6.6 Indicateurs de performance

F1 macro, rappel, matrice de confusion, latence.

2.6.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
equipment-service	Inventaire simulé des équipements et association aux signaux.	Entrée : équipement. Sortie : equipmentId, métadonnées.
telemetry-service	Import et stockage des signaux (séquences) ; consultation par équipement.	Entrée : séquence de mesures. Sortie : signalId.
anomaly-service	Demande la classification à ai-inference, enregistre classe/score et génère l'alerte si anomalie.	Entrée : signalId, modèle. Sortie : classe, score, alerte.
ai-inference (Python / FastAPI)	Charge CNN 1D et TCN, renvoie classe et probabilité.	Entrée : séquence normalisée. Sortie : classe, score, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

2.6.8 Limites et cadrage de faisabilité

Ne pas présenter une classification FordA comme une prédiction du temps avant panne.

2.6.9 Scénario de démonstration et livrables

Scénario : Charger une séquence, obtenir classe/score et historiser l'alerte.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Sujet 2.7 — [SAI-6] EnergyTwin Smart — Prévision intelligente de consommation électrique

Sujet également présent dans le Groupe 1 (fiche identique).

2.7.1 Problématique

Prévoir la consommation d'un bâtiment avec un service d'expérimentation et des alertes de dépassement.

2.7.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
UCI Appliances Energy Prediction	https://archive.ics.uci.edu/dataset/374/appliances+energy+prediction	19 735 observations, 28 variables, pas de 10 minutes.	Document initial
UCI Household Electric Power Consumption	https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption	Alternative/complément : consommation d'un foyer à la minute.	Ajout (à vérifier)

Code de départ

Ressource	Lien	Usage	Origine
Prévision météo (Keras)	https://keras.io/examples/timeseries/timeseries_weather_forecasting/	Pipeline LSTM de départ.	Document initial
Transformer séries temporelles (Keras)	https://keras.io/examples/timeseries/timeseries_classification_transformer/	Exemple de bloc d'attention.	Document initial
Couche Attention (Keras)	https://keras.io/api/layers/attention_layers/attention/	Documentation de la couche à intégrer au LSTM.	Ajout

2.7.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	LSTM	Prévision supervisée sur séquences
Avancée (imposée)	LSTM avec couche d'attention	Pondération des pas de temps pertinents

Concepts à comprendre : Prévision supervisée, séquences, couche attention, attribution de poids (sans interprétation causale).

2.7.4 Optimisation

- Comparer LSTM simple à LSTM + attention (couches Keras).
- 2 à 3 configurations (unités, fenêtre, learning rate).

2.7.5 Protocole expérimental

1. Découpage chronologique.
2. Mêmes variables et fenêtres pour les deux modèles.
3. Erreur par créneau horaire.
4. Seuil d'alerte défini sur validation.

2.7.6 Indicateurs de performance

MAE, RMSE, erreur selon créneau horaire, latence.

2.7.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
meter-service	Import CSV et consultation des séries de consommation ; contrôle de cohérence.	Entrée : CSV. Sortie : séries, historique.
energy-forecast-service	Appelle ai-inference, archive les prévisions et compare LSTM / LSTM-attention.	Entrée : fenêtre, modèle. Sortie : prévision archivée.
threshold-service	Gère les seuils de dépassement et enregistre les alertes.	Entrée : seuil, prévisions. Sortie : alertes.
ai-inference (Python / FastAPI)	Charge les deux modèles, renvoie prévision et, si disponible, poids d'attention.	Entrée : fenêtre normalisée. Sortie : prévision, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

2.7.8 Limites et cadrage de faisabilité

Un bâtiment, pas de véritable réseau électrique ; prédiction plutôt qu'optimisation du réseau.

2.7.9 Scénario de démonstration et livrables

Scénario : Importer un CSV, visualiser historique/prévisions et créer un seuil d'alerte.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Sujet 2.8 — [SAI-7] UrbanFlow Smart — Prévision du trafic par réseau de graphes

Sujet également présent dans le Groupe 3 (fiche identique).

2.8.1 Problématique

Prédire la vitesse du trafic à plusieurs points de mesure en exploitant l'information temporelle et les voisinages du réseau routier.

2.8.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
PeMSD7 (tutoriel Keras)	https://keras.io/examples/timeseries/timeseries_traffic_forecasting/	Sous-ensemble : distances et vitesses des capteurs.	Document initial
STGCN — dépôt d'origine de PeMSD7	https://github.com/VeritasYin/STGCN_IJCAI-18	Source des données PeMSD7 et du graphe.	Ajout (à vérifier)

Code de départ

Ressource	Lien	Usage	Origine
Exemple Keras trafic	https://keras.io/examples/timeseries/timeseries_traffic_forecasting/	Point de départ GCN-LSTM.	Document initial
PyTorch Geometric	https://github.com/pyg-team/pytorch_geometric	Bibliothèque de réseaux de graphes.	Document initial
PyTorch Geometric Temporal	https://github.com/benedekrozemberczki/pytorch_geometric_temporal	Modèles spatio-temporels prêts à l'emploi.	Ajout

2.8.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	LSTM par capteur	Série temporelle indépendante par capteur
Avancée (imposée)	Graph Convolution + LSTM (GCN-LSTM)	Agrégation des voisins + dynamique temporelle

Concepts à comprendre : Nœud = capteur ; arête = proximité ; agrégation des voisins ; LSTM sur le temps.

2.8.4 Optimisation

- Réduire à 20–50 capteurs si nécessaire.
- Entraîner les deux modèles sur splits temporels.
- Comparer avec et sans voisinage.

2.8.5 Protocole expérimental

1. Splits temporels identiques.
2. Graphe fixe fourni, construit une seule fois.
3. Erreurs par capteur.
4. Ablation : GCN-LSTM sans arêtes vs avec arêtes.

2.8.6 Indicateurs de performance

MAE, RMSE, erreurs par capteur ; comparaison avec et sans voisinage.

2.8.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
road-network-service	Gère capteurs et graphe (nœuds, arêtes, distances) ; expose le voisinage.	Entrée : capteurs, matrice d'adjacence. Sortie : graphe, voisins.
traffic-data-service	Import et consultation de l'historique des vitesses par capteur.	Entrée : séries de vitesses. Sortie : historique.
traffic-forecast-service	Appelle ai-inference, archive les prévisions et compare observé/prédit.	Entrée : capteur, horizon, modèle. Sortie : prévision archivée.
ai-inference (Python / FastAPI)	Charge LSTM et GCN-LSTM, reçoit fenêtre et graphe, renvoie les vitesses prédites.	Entrée : fenêtre + adjacence. Sortie : vitesses, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

2.8.8 Limites et cadrage de faisabilité

Graphe fourni et fixe ; ne pas imposer STGCN complet ni construction géographique complexe.

2.8.9 Scénario de démonstration et livrables

Scénario : Afficher le réseau simplifié, sélectionner un capteur et comparer vitesses observées/prédites.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Groupe 3 — Vision, imagerie et analyse spatiale

Sujets dont le cœur est l'image (drone, photo, satellite, médical) ou la structure spatiale (graphe de capteurs).

Code	Sujet
OLV-1	3.1 — Détection, comptage et géolocalisation des oliviers par drone
OLV-2	3.2 — Diagnostic précoce des maladies de l'olivier
OLV-4	3.3 — Analyse satellitaire multi-temporelle des oliveraies
OLV-6	3.4 — Détection temporelle des oliviers dépérissants ou morts
OLV-7	3.5 — Plateforme intelligente de comptage des oliviers et de reconnaissance de leurs variétés (IA + données géospatiales)
SAI-2	3.6 — MedRestore Smart — Restauration intelligente d'images
SAI-3	3.7 — MedVision Smart — Segmentation biomédicale intelligente
SAI-7	3.8 — UrbanFlow Smart — Prévision du trafic par réseau de graphes

Sujet 3.1 — [OLV-1] Détection, comptage et géolocalisation des oliviers par drone

3.1.1 Problématique

Détecter et compter les arbres, segmenter leurs couronnes et enregistrer des coordonnées géographiques robustes sur des sols et des plantations variés.

3.1.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
OliveTreeCrownsDb (Mendeley)	https://data.mendeley.com/datasets/xym8rd2srf/2	Détection et segmentation de couronnes d'oliviers ; pré-entraînement et benchmark.	Document initial
AgML olive_tree_crown_detection	https://huggingface.co/datasets/Project-AgML/olive_tree_crown_detection	Détection par boîtes englobantes, format prêt pour l'entraînement.	Document initial
UAV Tree Crown E1 (Roboflow)	https://universe.roboflow.com/edson-ihqiw/uav_tree_crown_e1	Transfert de méthodes aériennes (non spécifique olivier).	Document initial
Collecte Maroc	Données internes / à fournir	Orthomosaïque, images drone RGB, emprise de parcelle, résolution (cm/pixel), GPS, boîtes/polygones annotés, comptage terrain. Géré via dataset-service, MinIO et DVC.	À constituer

3.1.3 Algorithmes à comparer

Approche	Modèle	Rôle de comparaison
Baseline	Seuillage HSV/ExG + morphologie + composantes connexes	Point de comparaison non DL
Détection	YOLO détection	Rapidité et comptage
Instances	YOLO segmentation	Comptage + masque de couronne
Instances	Mask R-CNN	Qualité des limites de couronne
Forêt	DeepForest / Detectree2	Transfert de méthodes aériennes

3.1.4 Optimisation

- Optimiseurs : comparer SGD avec momentum et AdamW à budget d'époques constant pour les modèles compatibles ; ne pas l'imposer aux modèles pré-entraînés qui utilisent une recette propre.
- Réglages : résolution 640–1280, taille de tuile 512–1024, chevauchement 10–25 %, batch size selon VRAM, learning rate, weight decay, seuils de confiance et d'IoU du NMS.
- Augmentation : rotations, variation de luminosité, ombre, compression, blur ; vérifier la cohérence des masques et des coordonnées.
- Recherche d'hyperparamètres : Optuna TPE (20–40 essais) ou random search sur mAP/erreur de comptage en validation ; rapporter aussi latence et mémoire.
- Optimisation d'inférence : export ONNX, quantification INT8 après validation de la perte de précision, traitement par tuiles en batch.

3.1.5 Protocole expérimental

1. Préparer les orthomosaïques et convertir correctement les pixels en coordonnées du système de référence.
2. Annoter sur plusieurs exploitations ; splits géographiques stricts (toutes les tuiles d'une même parcelle dans le même split).

3. Évaluer la baseline, puis les détecteurs et segmentateurs avec un protocole identique.
4. Reconstruire les prédictions des tuiles, supprimer les doublons et tester le comptage par hectare.
5. Tester dans une région non vue et inspecter manuellement les faux positifs (ombres, autres arbres).

3.1.6 Indicateurs de performance

mAP@50 et @50:95, F1, IoU/Dice, MAE du comptage par parcelle, erreur GPS (m), secondes/hectare.

3.1.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
drone-ingestion-service	Reçoit les vols et images drone (multipart ou URL), valide format/MIME, extrait les métadonnées EXIF/XMP (GPS, altitude, résolution au sol), stocke les fichiers dans MinIO via media-service et publie un événement FlightRegistered.	Entrée : vols, images. Sortie : flightId, URI MinIO, événement.
orthomosaic-processing-service	Déclenche l'assemblage de l'orthomosaïque (worker conteneurisé, ex. OpenDroneMap) puis le tuilage (512–1024 px, chevauchement 10–25 %). Suit l'avancement via job-service, gère reprise et erreurs. Produit un GeoTIFF et un index de tuiles avec son système de coordonnées.	Entrée : JobRequested (URIs des images). Sortie : JobCompleted (URI GeoTIFF + tuiles).
tree-detection-service	Orchestre l'inférence par un worker Python (YOLO / Mask R-CNN / ONNX), reconstruit les prédictions de tuiles, applique le NMS inter-tuiles, convertit pixels → coordonnées géographiques et publie arbres/couronnes avec model_version et confidence.	Entrée : flightId, modelVersion. Sortie : arbres et couronnes normalisés, métriques du job.
tree-inventory-service	Persiste les arbres (point + polygone PostGIS), gère le dédoublonnage aux bordures, calcule le comptage par parcelle et par hectare, permet la validation humaine et exporte en GeoJSON.	Entrée : détections. Sortie : inventaire, GeoJSON, statistiques de comptage.

Worker IA associé (Python)

drone-detection-worker (Python / FastAPI / PyTorch ou ONNX Runtime) : prétraitement, inférence par tuile, post-traitement ; sans état, modèle versionné via model-registry-service.

Exemples de contrats REST

```
POST /api/drone/flights ; POST /api/drone/images
POST /api/tree-detections/jobs {flightId, modelVersion}
GET /api/tree-detections/jobs/{jobId} ; GET /api/parcels/{id}/trees
```

3.1.8 Risques et contrôles

Faux arbres dus aux ombres ; géoréférencement insuffisant ; duplication aux bordures de tuiles ; jeu de test biaisé.

3.1.9 Livrables

Dataset annoté, notebooks d'entraînement, 3–5 modèles comparés, API REST, carte des arbres et rapport d'erreurs.

Sujet 3.2 — [OLV-2] Diagnostic précoce des maladies de l'olivier

Sujet également présent dans le Groupe 1 (fiche identique).

3.2.1 Problématique

Classifier des symptômes phytosanitaires, localiser les lésions lorsque des masques sont disponibles et produire une alerte avec incertitude, sans remplacer l'expertise agronomique.

3.2.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
CNN olive dataset (GitHub)	https://github.com/sinanuguz/CNN_olive_dataset	Classification d'images de feuilles d'olivier saines/malades.	Document initial
Olive leaf disease (Kaggle, « zeytin »)	https://www.kaggle.com/datasets/serhathoca/zeytin	Classification de maladies foliaires ; comparaison de backbones.	Document initial
Olive tree hyperspectral stress (Zenodo)	https://zenodo.org/records/10336545	Détection de stress hydrique/biotique par signatures spectrales.	Document initial
Collecte Maroc	Données internes / à fournir	Feuille/fruit/arbre, classe validée par agronome, variété, région, saison, gravité, résolution, indicateur image_inexploitable ; masques de lésion pour la segmentation.	À constituer

3.2.3 Algorithmes à comparer

Approche	Modèle	Rôle de comparaison
Baseline	Descripteurs couleur/texture + SVM	Référence interprétable
Classification	ResNet18 / EfficientNet	Transfert léger et efficace
Classification	ConvNeXt-Tiny	Backbone convolutionnel moderne
Classification	ViT-Tiny / ViT-Small	Comparaison Transformer
Segmentation	U-Net / SegFormer	Localisation si masques disponibles

3.2.4 Optimisation

- Comparaison AdamW vs SGD momentum ; learning rate 1e-5 à 1e-3, weight decay, dégel progressif des couches, taille d'image et batch size.
- Pondération des classes, focal loss ou sampler équilibré, sans mélanger les images d'un même arbre entre splits.
- Early stopping sur macro-F1, ReduceLROnPlateau ou scheduler cosinus, warmup selon l'architecture.
- Optuna TPE / random search sur lr, dropout et augmentation ; 20–30 essais selon le budget GPU.
- Calibration des probabilités (temperature scaling) et seuil d'abstention (image inexploitable, faible confiance) défini sur validation.

3.2.5 Protocole expérimental

1. Faire relire les étiquettes par un agronome ; ne pas fusionner des maladies distinctes sous une étiquette vague.
2. Séparer entraînement/validation/test par arbre ou exploitation ; conserver une exploitation externe.
3. Comparer classification avec/sans transfer learning ; produire des matrices de confusion par classe et par variété.

4. Évaluer en conditions OOD (autres appareils, éclairages, régions) et mesurer les erreurs de confiance.
5. Valider le flux de confirmation humaine avant la création d'une alerte d'intervention.

3.2.6 Indicateurs de performance

Macro-F1, rappel par maladie, matrice de confusion, ROC-AUC si pertinent, ECE/calibration, taux d'abstention, Dice pour les lésions.

3.2.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
plant-image-service	Reçoit les photos (feuille, fruit, arbre), contrôle leur qualité (flou, exposition, résolution, MIME, antivirus), les associe à leur contexte (parcelle, variété, date) et les stocke dans MinIO. Marque les images inexploitables avant toute inférence.	Entrée : photo + contexte. Sortie : imageId, score qualité, URI MinIO.
disease-diagnosis-service	Orchestre la classification et, si besoin, la segmentation via un worker Python. Applique la calibration (temperature scaling) et le seuil d'abstention, puis renvoie top-k, probabilité calibrée, masque de lésion et statut (automatique / abstention). Historise model_version.	Entrée : imageId. Sortie : diagnostic, confiance, masque, statut.
agronomist-review-service	File de validation experte : l'agronome confirme ou corrige le diagnostic, la décision est historisée et les corrections sont renvoyées au dataset-service pour enrichir les futurs réentraînements.	Entrée : diagnosticId. Sortie : décision, annotation corrigée.
plant-health-alert-service	Crée les alertes phytosanitaires uniquement après confirmation humaine, calcule la priorité (gravité × surface × confiance), conserve l'historique par parcelle et déclenche les notifications via notification-service.	Entrée : diagnostic confirmé. Sortie : alerte priorisée, notification.

Worker IA associé (Python)

disease-inference-worker (Python / FastAPI / PyTorch) : classification, segmentation, calibration ; modèles ResNet/EfficientNet/ConvNeXt/ViT exportables en ONNX.

Exemples de contrats REST

```
POST /api/plant-images ; POST /api/disease-diagnoses
GET /api/disease-diagnoses/{id}
POST /api/agronomist-reviews/{diagnosisId}/decision
```

3.2.8 Risques et contrôles

Biais de fond/éclairage ; maladies absentes des datasets ; confusion stress/maladie ; surconfiance des scores.

3.2.9 Livrables

Classification comparée, détection des images non fiables, API d'analyse de photo, interface de validation agronome.

Sujet 3.3 — [OLV-4] Analyse satellitaire multi-temporelle des oliveraies

Sujet également présent dans le Groupe 2 (fiche identique).

3.3.1 Problématique

Détecter des ruptures ou anomalies de vigueur à l'échelle de la parcelle sur des séries optiques multi-dates, en tenant compte des nuages, des saisons et des pratiques culturales.

3.3.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
Sentinel-2 L2A (Earth Engine)	https://developers.google.com/earth-engine/datasets/catalog/COPERNICUS_S2_SR_HARMONIZED	Séries NDVI/NDRE/NDWI/SAVI à 10–20 m, masques QA.	Document initial
Landsat Collection 2 (USGS)	https://www.usgs.gov/landsat-missions/landsat-collection-2	Historique long (30 m) pour la baseline saisonnière.	Document initial
ERA5-Land (météo)	https://developers.google.com/earth-engine/datasets/catalog/ECMWF_ERA5_LAND_DAILY_AGGR	Variables météo pour la fusion et l'ablation.	Document initial
Copernicus Data Space Ecosystem	https://dataspace.copernicus.eu/	Accès alternatif aux scènes Sentinel hors Earth Engine.	Ajout
Collecte Maroc	Données internes / à fournir	Polygones géoréférencés, visites terrain datées, irrigation, observations de stress ; masques nuage et qualité satellite.	À constituer

3.3.3 Algorithmes à comparer

Approche	Modèle	Rôle de comparaison
Règles expertes	Seuil z-score saisonnier NDVI/NDRE	Baseline transparente
Non supervisé	Isolation Forest	Anomalies multivariées
Non supervisé	One-Class SVM	Frontière de normalité
Reconstruction	Autoencoder / LSTM-AE	Écarts à la saisonnalité
Supervisé	XGBoost + météo	Si anomalies terrain étiquetées

3.3.4 Optimisation

- Agrégation temporelle 5–15 jours, pourcentage de nuage maximal et filtres de qualité ; attention aux pixels mixtes.
- Isolation Forest : n_estimators, max_samples, contamination (réglée sans utiliser le test). One-Class SVM : gamma et nu.
- Autoencoder : AdamW, lr, latent_dim, longueur de fenêtre, reconstruction loss.
- Optimisation bayésienne/TPE sur validation temporelle et campagnes distinctes, avec budget d'essais contrôlé.
- Ablation : NDVI seul vs NDVI+NDRE+NDWI+SAVI vs fusion météo ; lisser sans masquer les alertes réellement brusques.

3.3.5 Protocole expérimental

1. Extraire les images via Earth Engine, appliquer les masques QA, fixer période et couverture par parcelle.
2. Calculer les séries NDVI/NDRE/NDWI/SAVI puis normaliser par saison et par parcelle.
3. Comparer règles expertes et 2–4 méthodes statistiques/ML ; mesurer la stabilité interannuelle.
4. Faire correspondre alertes et observations terrain avec une tolérance temporelle prédéfinie.
5. Tester un bassin ou une campagne jamais vue pour la robustesse spatiale et temporelle.

3.3.6 Indicateurs de performance

Précision, rappel, F1 des alertes, délai moyen de détection, faux positifs par parcelle, stabilité interannuelle.

3.3.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
satellite-acquisition-service	Orchestre l'extraction Earth Engine / Copernicus, sélectionne les scènes par parcelle et période, applique les masques nuage/QA et tient un catalogue de scènes avec leur couverture utile.	Entrée : import-job (parcelle, dates). Sortie : scènes filtrées, catalogue.
vegetation-index-service	Calcule NDVI, NDRE, NDWI, SAVI par parcelle (statistiques zonales), agrège par fenêtre de 5–15 jours, normalise par saison et archive les séries.	Entrée : scènes. Sortie : séries d'indices par parcelle.
anomaly-detection-service	Lance les détecteurs (z-score, Isolation Forest, One-Class SVM, autoencoder) via un worker Python et renvoie des anomalies datées avec score et variable explicative.	Entrée : séries d'indices. Sortie : anomalies datées, scores, interprétation.
parcel-monitoring-service	Cartes temporelles, courbes de vigueur, saisie des validations terrain, appariement alerte/observation avec tolérance temporelle et calcul des KPI (faux positifs, délai de détection).	Entrée : anomalies, observations terrain. Sortie : tableau de bord, KPI.

Worker IA associé (Python)

satellite-analytics-worker (Python / FastAPI / scikit-learn / PyTorch, rasterio, Earth Engine API) : calcul d'indices, détection d'anomalies, entraînement des autoencodeurs.

Exemples de contrats REST

```
POST /api/satellite/import-jobs {parcelId, startDate, endDate}
GET /api/parcels/{id}/vegetation-series
POST /api/anomalies/analyze ; GET /api/parcels/{id}/anomalies
```

3.3.8 Risques et contrôles

Nuages résiduels ; résolution satellite insuffisante pour les arbres individuels ; fausses ruptures dues à la phénologie.

3.3.9 Livrables

Pipeline géospatial reproductible, modèles comparés, couches cartographiques et tableau de bord d'alertes datées.

Sujet 3.4 — [OLV-6] Détection temporelle des oliviers dépérissants ou morts

Sujet également présent dans le Groupe 1 (fiche identique).

3.4.1 Problématique

Suivre un même arbre à plusieurs dates, quantifier les changements de couronne et de vigueur et identifier les individus nécessitant une inspection.

3.4.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
OQDS Insight (Data in Brief)	https://doi.org/10.1016/j.dib.2025.111615	Données de qualité/état d'oliviers ; référence pour le suivi.	Document initial
NEON tree crowns DTA (Hugging Face)	https://huggingface.co/datasets/weecology/neon-tree-crowns-dta	Couronnes d'arbres multi-dates (autres essences) ; transfert méthodologique.	Document initial
Olive hyperspectral stress (Zenodo)	https://zenodo.org/records/10336545	Signatures spectrales de stress.	Document initial
OliveTreeCrownsDb (Mendeley)	https://data.mendeley.com/datasets/xym8rd2srf/2	Référence initiale non temporelle pour les couronnes.	Document initial
Collecte Maroc	Données internes / à fournir	tree_id stable, orthomosaïques répétées, couronnes, date, indices de vigueur, statut confirmé sain/stressé/mort.	À constituer

3.4.3 Algorithmes à comparer

Approche	Modèle	Rôle de comparaison
Baseline	Seuil de variation NDVI/couronne	Approche explicable
Statistique	CUSUM ou détection de rupture	Détection chronologique
Anomalie	Isolation Forest	Variables de changement
Siamois	Siamese CNN deux dates	Similarité visuelle
Temps	Autoencoder temporel / TCN	Séquences nombreuses et alignées

3.4.4 Optimisation

- Optimiser en premier le recalage : points homologues, erreur géométrique, résolution commune, stabilité des tree_id.
- Siamese : contrastive/triplet loss, AdamW, learning rate, équilibrage des paires positives/négatives.
- Séries : longueur de fenêtre, seuil d'alerte, persistance temporelle et filtrage des changements saisonniers.
- Ruptures/statistiques : seuils et pénalité calibrés uniquement en validation ; Optuna TPE si l'espace est suffisant.
- Modèle léger ONNX pour l'inférence à l'échelle de l'arbre ; mesure du coût par 1 000 arbres.

3.4.5 Protocole expérimental

1. Constituer des couples et séquences d'observation des mêmes arbres avec co-enregistrement.
2. Contrôler l'identité des arbres entre saisons, les changements de couronne et les doublons possibles.
3. Comparer seuils experts, rupture statistique, Isolation Forest et Siamese si les exemples étiquetés suffisent.
4. Évaluer sur exploitations non vues et campagnes postérieures, sans paire partageant les mêmes arbres entre splits.

5. Mettre en place l'inspection agronomique et l'historique des décisions avant toute alerte définitive.

3.4.6 Indicateurs de performance

Rappel des arbres dépérissants confirmés, précision, délai avant validation terrain, alertes répétées, erreur de correspondance.

3.4.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
tree-tracking-service	Maintient l'identité (tree_id) de chaque arbre entre acquisitions : appariement spatial (distance, IoU des couronnes), gestion des fusions, scissions et doublons, matrice de correspondance entre dates.	Entrée : observations d'arbres. Sortie : tree_id stable, correspondances.
change-analysis-service	Recalage des orthomosaïques (points homologues), extraction des différences de couronne et de vigueur, calcul des indicateurs de changement via un worker Python.	Entrée : parcelle, dateFrom, dateTo. Sortie : indicateurs de changement par arbre.
decline-risk-service	Calcule le score de dépérissement par arbre, applique seuils, persistance temporelle et filtrage saisonnier, et tient la chronologie ; classe l'arbre en « candidat à inspecter ».	Entrée : indicateurs. Sortie : score, statut, chronologie.
field-inspection-service	Missions de terrain (liste d'arbres à inspecter, itinéraire), photos, confirmation sain/stressé/mort, historique des décisions et retour des labels au dataset-service.	Entrée : candidats. Sortie : confirmation, photos, labels.

Worker IA associé (Python)

change-detection-worker (Python / FastAPI / PyTorch / OpenCV / ruptures) : recalage, Siamese CNN, détection de rupture, export ONNX.

Exemples de contrats REST

```
POST /api/trees/{id}/observations
POST /api/tree-changes/jobs {parcelId, dateFrom, dateTo}
GET /api/trees/{id}/health-history ; POST /api/inspections/{id}/confirm
```

3.4.8 Risques et contrôles

Jeux externes insuffisants pour apprendre tous les scénarios marocains ; recalage incorrect = fausse alerte ; nécessité d'observations répétées.

3.4.9 Livrables

Suivi multi-dates d'arbres individuels, comparaison de 3–4 algorithmes, carte des évolutions et workflow d'inspection.

Sujet 3.5 — [OLV-7] Plateforme intelligente de comptage des oliviers et de reconnaissance de leurs variétés (IA + données géospatiales)

3.5.1 Problématique

Comment automatiser le comptage des oliviers dans une parcelle, identifier leurs variétés et exploiter leurs positions géographiques afin d'améliorer le suivi agricole, la gestion de l'irrigation et la productivité des exploitations oléicoles marocaines ? Objectifs : détecter et compter par YOLO, géolocaliser, classifier les variétés (feuilles, fruits, branches), produire un score de confiance, intégrer météo et satellite, proposer une interface cartographique et des indicateurs agricoles.

3.5.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
OliveTreeCrownsDb (Mendeley)	https://data.mendeley.com/datasets/xym8rd2srf/2	Comptage et segmentation des couronnes (réutilisé du Sujet 1).	Réutilisé (S1)
AgML olive_tree_crown_detection	https://huggingface.co/datasets/Project-AgML/olive_tree_crown_detection	Détection par boîtes (réutilisé du Sujet 1).	Réutilisé (S1)
Olive leaves — cultivars (IPB, DOI 10.34620/dadosipb/TVYE8K)	https://doi.org/10.34620/dadosipb/TVYE8K	Classification de cultivars à partir de feuilles (environ 4 200 images, 4 catégories d'après l'article associé).	Ajout (à vérifier)
Olive-Fruit Variety Classification (IEEE Access 2019)	https://doi.org/10.1109/ACCESS.2019.2947160	Article de référence : 2 800 fruits, 7 variétés ; disponibilité des images à confirmer auprès des auteurs.	Ajout (à vérifier)
Image-Based Classification of Olive Varieties Native to Türkiye (arXiv 2602.18530)	https://arxiv.org/abs/2602.18530	Référence de comparaison ResNet50/EfficientNet/ViT/ConvNeXt (2 500 images, 5 variétés) ; lien du dataset à chercher dans l'article.	Ajout (à vérifier)
Sentinel-2 L2A (Earth Engine)	https://developers.google.com/earth-engine/datasets/catalog/COPERNICUS_S2_SR_HARMONIZED	Indicateurs de santé/vigueur de la parcelle.	Réutilisé (S4)
NASA POWER / ERA5-Land	https://power.larc.nasa.gov/	Météo pour les indicateurs d'irrigation.	Réutilisé (S3)
Collecte Maroc	Données internes / à fournir	Images aériennes d'oliveraies marocaines (drone, orthomosaïques), annotations des couronnes, GPS ; images rapprochées de feuilles/fruits/branches avec variété vérifiée par un spécialiste (ex. Picholine marocaine, Haouzia, Menara). Les images aériennes seules ne garantissent pas une reconnaissance fiable des variétés.	À constituer

3.5.3 Algorithmes à comparer

Approche	Modèle	Rôle de comparaison
Baseline (alignement stratégie commune)	Seuillage HSV/ExG (comptage) ; descripteurs couleur/texture + SVM (variété)	Point de comparaison non DL
Comptage	YOLO détection vs Faster R-CNN	Rapidité vs précision du comptage
Couronnes	YOLO segmentation vs Mask R-CNN	Qualité des masques de couronne
Variétés	ResNet50 vs EfficientNet vs Vision Transformer	Reconnaissance fine des variétés
Incertitude	Classe « variété incertaine » + vérification humaine	Sécurité et fiabilité

3.5.4 Optimisation

- Optimiseurs : Adam, AdamW et SGD pour les réseaux.
- Hyperparamètres : Optuna ou Random Search.
- Augmentation de données, transfert d'apprentissage, calibration des probabilités.
- Export ONNX pour l'inférence.

3.5.5 Protocole expérimental

1. Harmonisé sur les Sujets 1 et 2 : splits stricts par parcelle et par exploitation (comptage), par arbre ou exploitation (variétés).
2. Évaluer baseline, puis détecteurs, segmentateurs et classifieurs avec un protocole identique.
3. Définir le seuil de la classe « variété incertaine » sur validation ; vérification humaine des cas incertains.
4. Tester sur une exploitation et une région non vues ; mesurer latence p95 et débit API.

3.5.6 Indicateurs de performance

mAP@50:95, précision/rappel, F1, erreur de comptage par parcelle, IoU/Dice, exactitude et macro-F1 des variétés, taux de prédictions incertaines, latence p95, débit API.

3.5.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
api-gateway	Spring Cloud Gateway : point d'entrée unique, routage, limitation de débit, validation des jetons (socle commun).	Entrée : requêtes clients. Sortie : requêtes routées et sécurisées.
authentication-service	Authentification et autorisation (Keycloak / OIDC), rôles et droits par exploitation (socle commun identity-service).	Entrée : identifiants. Sortie : jetons JWT.
orchard-tree-service	Gestion des exploitations, parcelles et arbres dans PostGIS ; exports GeoJSON ; référentiel partagé par tous les sujets.	Entrée : arbres détectés. Sortie : inventaire, GeoJSON.
tree-detection-service	Spring Boot + moteur Python/FastAPI : lance les jobs de détection/segmentation par RabbitMQ ou Kafka, assemble les résultats et publie arbres et couronnes (même service que le Sujet 1).	Entrée : job de détection. Sortie : arbres, couronnes, confiance.
variety-classification-service	Classifie les variétés (ResNet50/EfficientNet/ViT) à partir de feuilles, fruits ou branches, renvoie variété, score calibré et statut « variété incertaine » avec file de vérification humaine.	Entrée : images rapprochées. Sortie : variété, confiance, statut.
satellite-service	Fournit les indices de végétation Sentinel-2 par parcelle (réutilise les Sujets 4 et 6).	Entrée : parcelle, période. Sortie : séries d'indices.
weather-service	Fournit météo et prévisions par parcelle (réutilise weather-data-service du Sujet 3).	Entrée : parcelle. Sortie : séries météo.
analytics-service	Agrège nombre d'oliviers, répartition par variété, niveaux de confiance, arbres incertains et indicateurs météo/irrigation/santé pour le tableau de bord.	Entrée : événements métier. Sortie : indicateurs agrégés.
notification-service	Alertes et notifications (arbres incertains, jobs terminés) ; socle commun.	Entrée : événements. Sortie : notifications.

Worker IA associé (Python)

olive-vision-worker (Python / FastAPI / PyTorch / YOLO) : détection, segmentation et classification des variétés ; export ONNX, modèles versionnés via MLflow.

Exemples de contrats REST

```
POST /api/detections/jobs ; GET /api/detections/jobs/{id}
GET /api/orchards/{id}/trees ; GET /api/orchards/{id}/geojson
```

```
POST /api/varieties/predict ; GET /api/orchards/{id}/variety-distribution
GET /api/orchards/{id}/analytics
```

3.5.8 Risques et contrôles

Images aériennes insuffisantes pour distinguer les variétés ; variétés visuellement proches ; faux arbres (ombres) ; surconfiance des scores ; fuites de données entre parcelles.

3.5.9 Livrables

Plateforme OliveSmart affichant le nombre total d'oliviers, la répartition par variété et les niveaux de confiance, les arbres incertains, les positions sur carte et des indicateurs météo, irrigation et santé des cultures.
Technologies : Java, Spring Boot, Spring Cloud Gateway, Spring Security, PostgreSQL/PostGIS, Python, FastAPI, PyTorch, YOLO, React/TypeScript, Leaflet, MinIO, Docker, GitHub Actions.

Sujet 3.6 — [SAI-2] MedRestore Smart — Restauration intelligente d'images

3.6.1 Problématique

Restaurer des images médicales ou biomédicales bruitées en entraînant deux réseaux supervisés sur des paires image propre / image altérée.

3.6.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
MedMNIST (sous-ensemble 2D)	https://github.com/MedMNIST/MedMNIST	Images biomédicales 2D ; bruit gaussien ajouté seulement aux entrées. Banc d'essai pédagogique, pas une validation clinique.	Document initial
MedMNIST — site officiel	https://medmnist.com/	Documentation, tailles d'images et téléchargement des sous-ensembles.	Ajout

Code de départ

Ressource	Lien	Usage	Origine
segmentation_models.pytorch	https://github.com/qubvel-org/segmentation_models.pytorch	U-Net et encodeurs pré-entraînés.	Document initial
DnCNN (référence originale)	https://github.com/cszn/DnCNN	Architecture de débruitage résiduel à adapter en petit DnCNN.	Ajout

3.6.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	CNN résiduel de débruitage (petit DnCNN)	Apprend le bruit résiduel à soustraire
Avancée (imposée)	U-Net léger	Encodeur-décodeur avec skip connections

Concepts à comprendre : Convolution, connexions résiduelles, encodeur-décodeur, skip connections.

3.6.4 Optimisation

- Former des paires propres/bruitées.
- Entraîner les deux architectures à résolution 64–128 px.
- Ajuster taux d'apprentissage et intensité du bruit.

3.6.5 Protocole expérimental

1. Split train/validation/test avant tout ajout de bruit ; bruit appliqué aux entrées uniquement.
2. Mêmes images et mêmes niveaux de bruit pour les deux modèles.
3. Tester au moins deux intensités de bruit, dont une non vue à l'entraînement.
4. Visualiser les cartes d'erreur.

3.6.6 Indicateurs de performance

PSNR, SSIM, MSE, temps de prédiction, visualisation des erreurs.

3.6.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
image-service	Dépôt, validation (format, taille, MIME) et stockage des images ; renvoie un identifiant d'image.	Entrée : image. Sortie : imageId, URI de stockage.
restoration-service	Gère les demandes de restauration : modèle choisi, appel à ai-inference, enregistrement de l'image restaurée.	Entrée : imageId, modèle. Sortie : restorationId, image restaurée.
comparison-service	Calcule PSNR/SSIM/MSE entre originale, bruitée et restaurée ; garde l'historique des comparaisons.	Entrée : restorationId. Sortie : métriques, historique.
ai-inference (Python / FastAPI)	Charge DnCNN et U-Net légers, restaure l'image à la résolution d'entraînement.	Entrée : image prétraitée. Sortie : image restaurée, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

3.6.8 Limites et cadrage de faisabilité

Images 2D seulement ; aucune promesse de diagnostic ni de reconstruction IRM physique.

3.6.9 Scénario de démonstration et livrables

Scénario : Importer une image altérée, choisir le modèle, afficher originale/restaurée et métriques.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Sujet 3.7 — [SAI-3] MedVision Smart — Segmentation biomédicale intelligente

Sujet également présent dans le Groupe 1 (fiche identique).

3.7.1 Problématique

Développer une plateforme de segmentation d'images avec entraînement, comparaison et contrôle humain des masques.

3.7.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
BUSI — Breast Ultrasound Images (Kaggle)	https://www.kaggle.com/datasets/aryashah2k/breast-ultrasound-images-dataset	À utiliser si masques et licence sont validés.	Document initial
Kvasir-SEG	https://datasets.simula.no/kvasir-seg/	Alternative : segmentation binaire à masques vérifiés.	Ajout (à vérifier)

Code de départ

Ressource	Lien	Usage	Origine
segmentation_models.pytorch	https://github.com/qubvel-org/segmentation_models.pytorch	U-Net et U-Net++ avec encodeurs pré-entraînés.	Document initial
UNet++ (référence originale)	https://github.com/MrGiovanni/UNetPlusPlus	Implémentation de l'article d'origine.	Ajout

3.7.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	U-Net	Segmentation binaire par encodeur-décodeur
Avancée (imposée)	U-Net++	Skip connections denses imbriquées

Concepts à comprendre : Encodeur-décodeur, skip connections, masque de segmentation, Dice.

3.7.4 Optimisation

- Entraîner U-Net et U-Net++ sur images 128–256 px.
- Tester deux fonctions de perte (ex. BCE+Dice vs Dice) ou deux jeux d'hyperparamètres.

3.7.5 Protocole expérimental

1. Split par patient/image, sans fuite entre train et test.
2. Mêmes partitions et même prétraitement pour les deux modèles.
3. Analyse d'échecs : cas où les masques divergent.
4. Validation humaine d'une sortie dans l'interface.

3.7.6 Indicateurs de performance

Dice, IoU, rappel, coût d'inférence ; analyse d'échecs.

3.7.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
exam-image-service	Gère images et métadonnées fictives (aucune donnée identifiante) ; stockage et consultation.	Entrée : image + métadonnées. Sortie : examId, URI.

Microservice	Description et responsabilité	Entrées → sorties
segmentation-service	Lance les tâches d'inférence pour U-Net et U-Net++, enregistre les masques et métriques.	Entrée : examId, modèle. Sortie : masques par modèle.
mask-review-service	Visualise et compare les masques, permet corrections et validation par un humain, avec historique.	Entrée : masque, décision. Sortie : masque validé/corrigé.
ai-inference (Python / FastAPI)	Charge U-Net et U-Net++, prétraite l'image et renvoie les masques binaires.	Entrée : image. Sortie : masque, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

3.7.8 Limites et cadrage de faisabilité

Jeu de données réduit ; pas de données identifiantes ; résultats à visée pédagogique uniquement.

3.7.9 Scénario de démonstration et livrables

Scénario : Charger une image, obtenir deux masques, comparer et valider une sortie.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Sujet 3.8 — [SAI-7] UrbanFlow Smart — Prévision du trafic par réseau de graphes

Sujet également présent dans le Groupe 2 (fiche identique).

3.8.1 Problématique

Prédire la vitesse du trafic à plusieurs points de mesure en exploitant l'information temporelle et les voisinages du réseau routier.

3.8.2 Jeux de données et liens

Dataset	Lien	Usage dans le projet	Origine
PeMSD7 (tutoriel Keras)	https://keras.io/examples/timeseries/timeseries_traffic_forecasting/	Sous-ensemble : distances et vitesses des capteurs.	Document initial
STGCN — dépôt d'origine de PeMSD7	https://github.com/VeritasYin/STGCN_IJCAI-18	Source des données PeMSD7 et du graphe.	Ajout (à vérifier)

Code de départ

Ressource	Lien	Usage	Origine
Exemple Keras trafic	https://keras.io/examples/timeseries/timeseries_traffic_forecasting/	Point de départ GCN-LSTM.	Document initial
PyTorch Geometric	https://github.com/pyg-team/pytorch_geometric	Bibliothèque de réseaux de graphes.	Document initial
PyTorch Geometric Temporal	https://github.com/benedekrozemberczki/pytorch_geometric_temporal	Modèles spatio-temporels prêts à l'emploi.	Ajout

3.8.3 Modèles à comparer

Rubrique	Modèle	Rôle de comparaison
Référence	LSTM par capteur	Série temporelle indépendante par capteur
Avancée (imposée)	Graph Convolution + LSTM (GCN-LSTM)	Agrégation des voisins + dynamique temporelle

Concepts à comprendre : Nœud = capteur ; arête = proximité ; agrégation des voisins ; LSTM sur le temps.

3.8.4 Optimisation

- Réduire à 20–50 capteurs si nécessaire.
- Entraîner les deux modèles sur splits temporels.
- Comparer avec et sans voisinage.

3.8.5 Protocole expérimental

1. Splits temporels identiques.
2. Graphe fixe fourni, construit une seule fois.
3. Erreurs par capteur.
4. Ablation : GCN-LSTM sans arêtes vs avec arêtes.

3.8.6 Indicateurs de performance

MAE, RMSE, erreurs par capteur ; comparaison avec et sans voisinage.

3.8.7 Microservices à mettre en place

Microservice	Description et responsabilité	Entrées → sorties
road-network-service	Gère capteurs et graphe (nœuds, arêtes, distances) ; expose le voisinage.	Entrée : capteurs, matrice d'adjacence. Sortie : graphe, voisins.
traffic-data-service	Import et consultation de l'historique des vitesses par capteur.	Entrée : séries de vitesses. Sortie : historique.
traffic-forecast-service	Appelle ai-inference, archive les prévisions et compare observé/prédit.	Entrée : capteur, horizon, modèle. Sortie : prévision archivée.
ai-inference (Python / FastAPI)	Charge LSTM et GCN-LSTM, reçoit fenêtre et graphe, renvoie les vitesses prédites.	Entrée : fenêtre + adjacence. Sortie : vitesses, latence.

Exemples de contrats REST

```
POST /api/data/import ; POST /api/predictions
GET /api/predictions/{id} ; GET /api/observations/{id}
```

3.8.8 Limites et cadrage de faisabilité

Graphe fourni et fixe ; ne pas imposer STGCN complet ni construction géographique complexe.

3.8.9 Scénario de démonstration et livrables

Scénario : Afficher le réseau simplifié, sélectionner un capteur et comparer vitesses observées/prédites.

Livrables : Notebook comparatif, courbes d'apprentissage et métriques ; modèles sauvegardés ; services Spring Boot et FastAPI ; interface ; tests REST ; Docker Compose ; courte présentation du modèle et de son intégration.

Partie D — Annexes (OliveSmart)

D.1 Correspondance avec les sept projets : DevOps et DevSecOps

Projet IA OliveSmart	DevOps	DevSecOps
1. Détection et comptage des oliviers	D1, D3, D4	S1, S4
2. Diagnostic des maladies	D2, D4	S2, S4
3. Irrigation intelligente	D2, D5	S2, S5
4. Analyse satellitaire	D1, D3, D4	S3, S4
5. Prévision du rendement	D2, D4	S4, S5
6. Détection du dépérissement	D2, D4, D5	S2, S5
7. Comptage et reconnaissance des variétés	D1, D3, D4	S1, S2, S4, S5

Remarque : les codes D1–D5 et S1–S5 ne sont pas définis dans le document source ; ils renvoient aux sujets DevOps/DevSecOps du programme et doivent y être précisés. Ils ne doivent pas être confondus avec les sujets S1–S7 du présent cahier des charges.

D.2 Découpage recommandé des équipes

Pour un master de 12 mois, chaque binôme développe 2–4 microservices métier et réutilise le socle commun. Construire d'abord un MVP en monorepo avec Docker Compose. Un microservice doit correspondre à une responsabilité métier réelle, pas à chaque classe Java ; éviter de multiplier les services.

Période	Résultat attendu
M1–M2	État de l'art, dataset, contrats API, critères de test.
M3–M4	Annotations, baseline, socle Spring Boot, schéma PostGIS.
M5–M6	Comparatifs de modèles, traçabilité MLflow, première API d'inférence.
M7–M8	Optimisation et validation hors échantillon, gestion des erreurs.
M9–M10	Services métier, tableau de bord, CI/CD et intégration.
M11–M12	Validation terrain, expérimentation finale et mémoire/article.

D.3 Références techniques OliveSmart

Ressource	Lien
Spring Boot	https://spring.io/projects/spring-boot
Spring Cloud Gateway	https://spring.io/projects/spring-cloud-gateway
Spring AI (facultatif : ne remplace pas un pipeline de vision PyTorch)	https://docs.spring.io/spring-ai/reference/
MLflow Serving	https://mlflow.org/docs/latest/genai/serving
Optuna	https://optuna.readthedocs.io/en/stable/tutorial/10_key_features/003_efficient_optimization_algorithms.html
PostGIS	https://postgis.net/
MinIO	https://min.io/
DVC	https://dvc.org/

D.4 Références techniques SmartAI

Ressource	Lien
Keras Examples	https://keras.io/examples/
PyTorch Geometric	https://github.com/pyg-team/pytorch_geometric
Segmentation Models PyTorch	https://github.com/qubvel-org/segmentation_models.pytorch